

Computer Architecture

(EE 371 / CE 321 / CS 330)

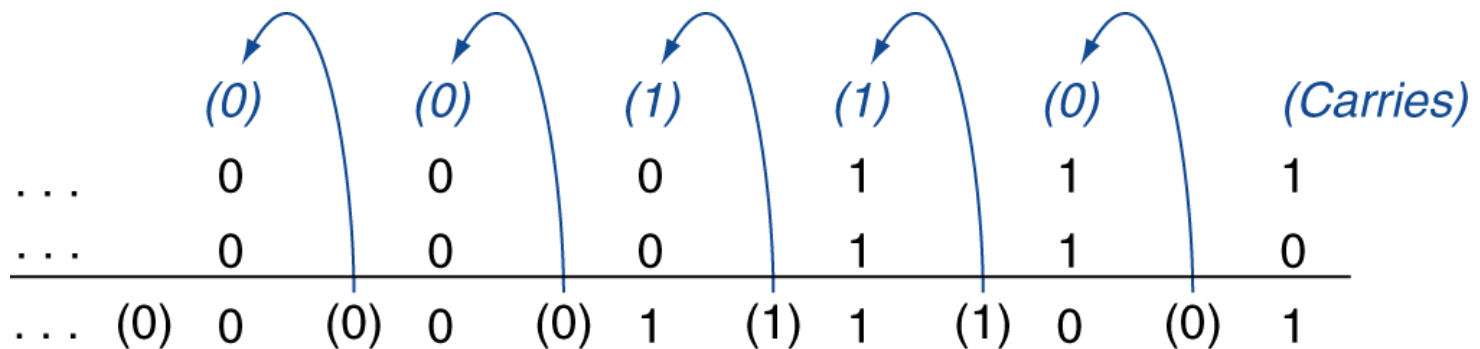
Dr. Farhan Khan
Assistant Professor,
Dhanani School of Science & Engineering,
Habib University

Outline: Unit 3

- Arithmetic for Computers: Review
- Digital Logic Design: Review
- Arithmetic Logic Unit (ALU)
- Single-Cycle Processor Implementation

Integer Addition

$$\begin{array}{r}
 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{\text{two}} - 7_{\text{ten}} \\
 +\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000110_{\text{two}} - 6_{\text{ten}} \\
 \hline
 -\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00001101_{\text{two}} - 13_{\text{ten}}
 \end{array}$$



Integer Addition: Overflow

$$\begin{array}{r} 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{\text{two}} - 7_{\text{ten}} \\ +\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000110_{\text{two}} - 6_{\text{ten}} \\ \hline -\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00001101_{\text{two}} - 13_{\text{ten}} \end{array}$$

Overflow occurs when the result from an operation cannot be represented with the available hardware, in this case a 64-bit word

- Adding +ve and –ve operands
 - Overflow is not possible
- Adding two +ve number
 - Overflow has occurred if the sign bit of the result is 1
- Adding two –ve numbers
 - Overflow has occurred if the sign bit of the result is 0

Integer Subtraction

- Subtraction through Addition
 - Add first operand and negation of the second operand
 - Example: $7 - 6 = 7 + (-6)$

$$\begin{array}{r} 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{\text{two}} - 7_{\text{ten}} \\ +\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111010_{\text{two}} - -6_{\text{ten}} \\ \hline =\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000001_{\text{two}} - 1_{\text{ten}} \end{array}$$

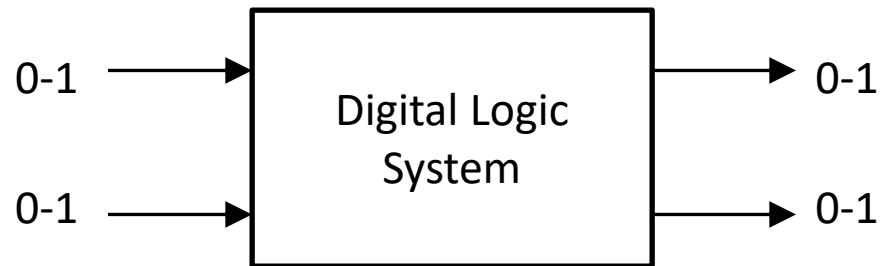
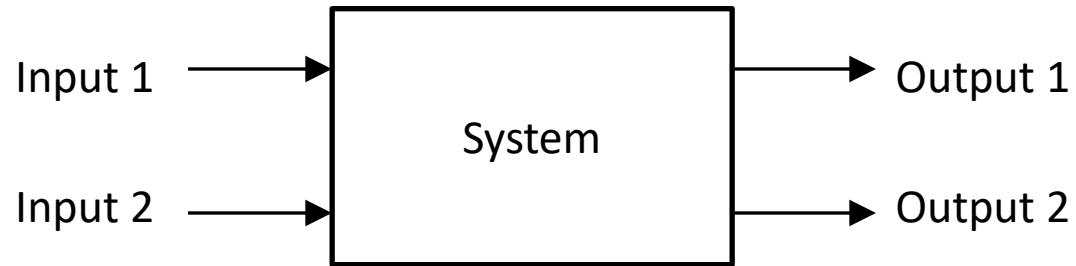
Integer Subtraction: Overflow

$$\begin{array}{r} 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{\text{two}} - 7_{\text{ten}} \\ +\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111010_{\text{two}} - -6_{\text{ten}} \\ \hline =\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000001_{\text{two}} - 1_{\text{ten}} \end{array}$$

Overflow occurs when the result from an operation cannot be represented with the available hardware, in this case a 64-bit word

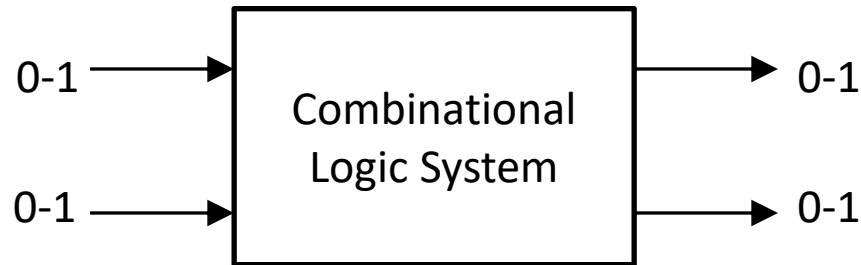
- Subtracting two +ve or two –ve operands
 - Overflow is not possible
- Subtracting +ve from -ve operand
 - Overflow has occurred if the sign bit of the result is 0
- Subtracting –ve from +ve operands
 - Overflow has occurred if the sign bit of the result is 1

Review of Logic Design

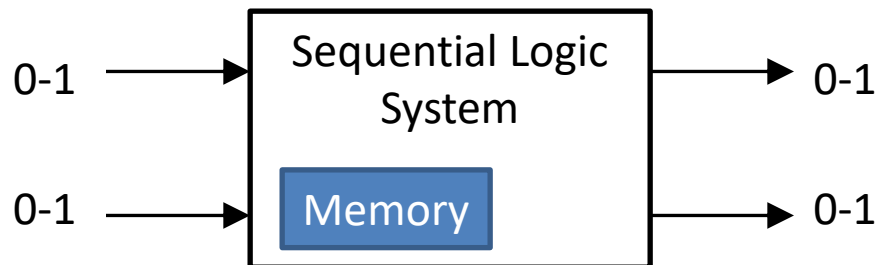


Review of Logic Design

- 2 Types of Logic Systems



The output of a combinational logic system depends only on the current input.



The output of a sequential logic system can depend both on the inputs and internal memory.

Review of Logic Design

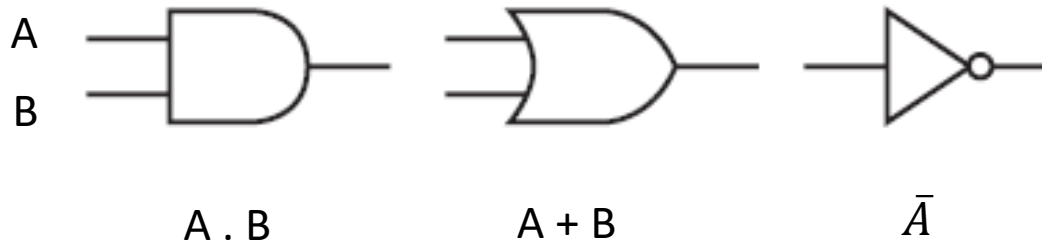
- Truth Table

- Because a combinational logic block contains no memory, it can be completely specified by defining the values of the outputs for each possible set of input values.
- Such a description is normally given as a truth table.

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

Review of Logic Design

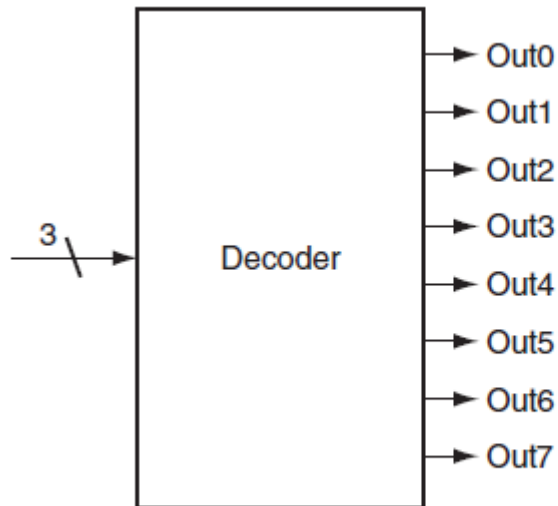
- Logic Gates
 - Logic blocks/systems are built from Logic Gates that implement basic logic functions



Review of Logic Design

- Decoder

- The decoder has an n-bit input and 2^n outputs, where only one output is asserted for each input combination



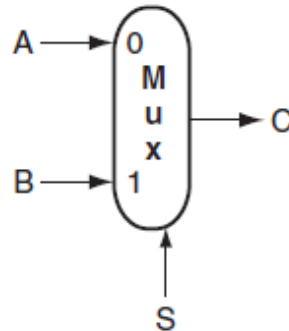
a. A 3-bit decoder

Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

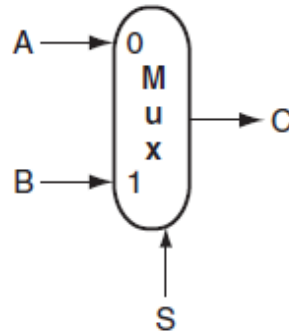
Review of Logic Design

- Multiplexor (Selector)
 - Consider the two-input multiplexor.
 - This multiplexor has three inputs: two data values and a selector (or control) value.
 - The selector value determines which of the inputs becomes the output.



Review of Logic Design

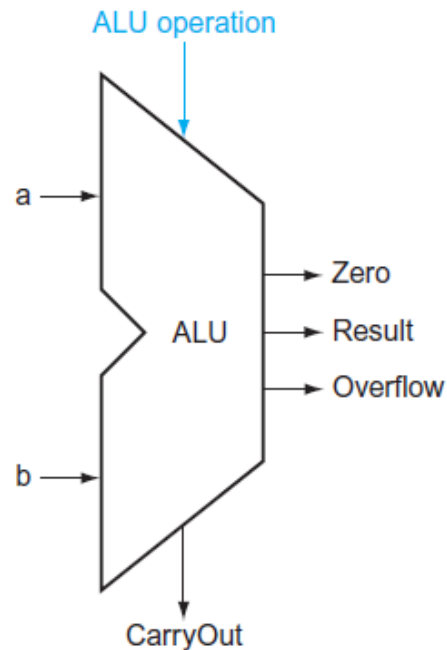
- Multiplexor (Selector)
 - Consider the two-input multiplexor.
 - This multiplexor has three inputs: two data values and a selector (or control) value.
 - The selector value determines which of the inputs becomes the output.



- Multiplexors can be created with an arbitrary number of data inputs.

Arithmetic Logic Unit (ALU)

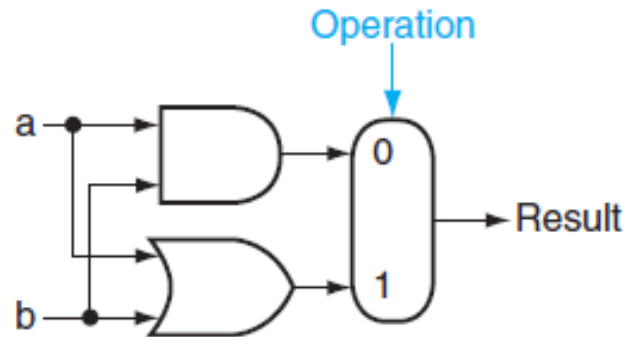
- The arithmetic logic unit (ALU) is the brain of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR.



Arithmetic Logic Unit (ALU)

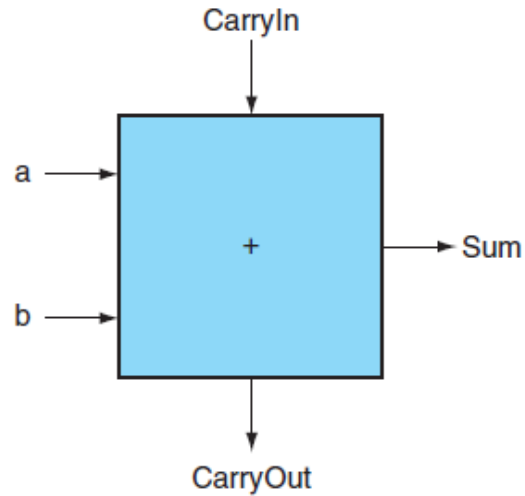
- The arithmetic logic unit (ALU) is the brain of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR.
- Let's focus on 1-bit ALU and progressively build an ALU with more functionality

Arithmetic Logic Unit (ALU)



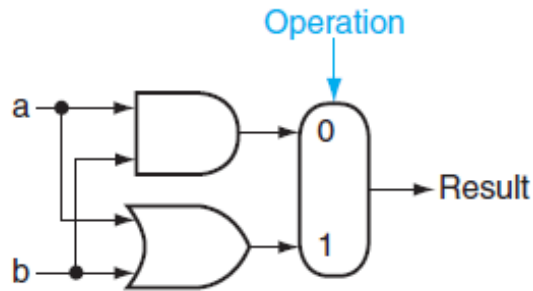
The 1-bit Logical Unit for AND and OR.

1-bit Adder

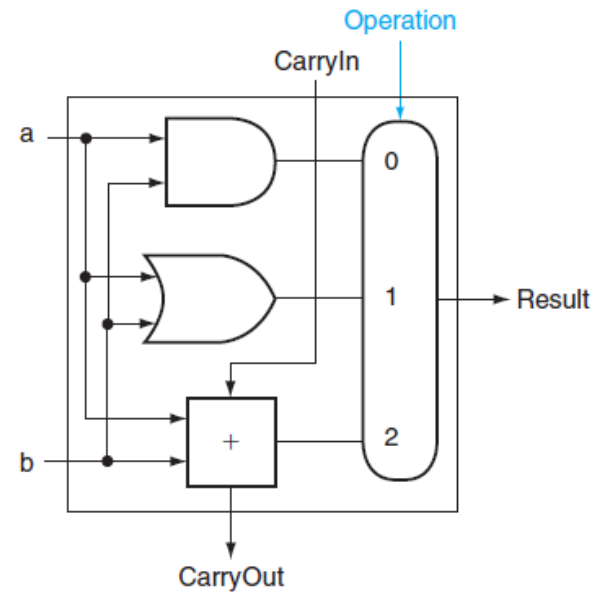


Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

Arithmetic Logic Unit (ALU)

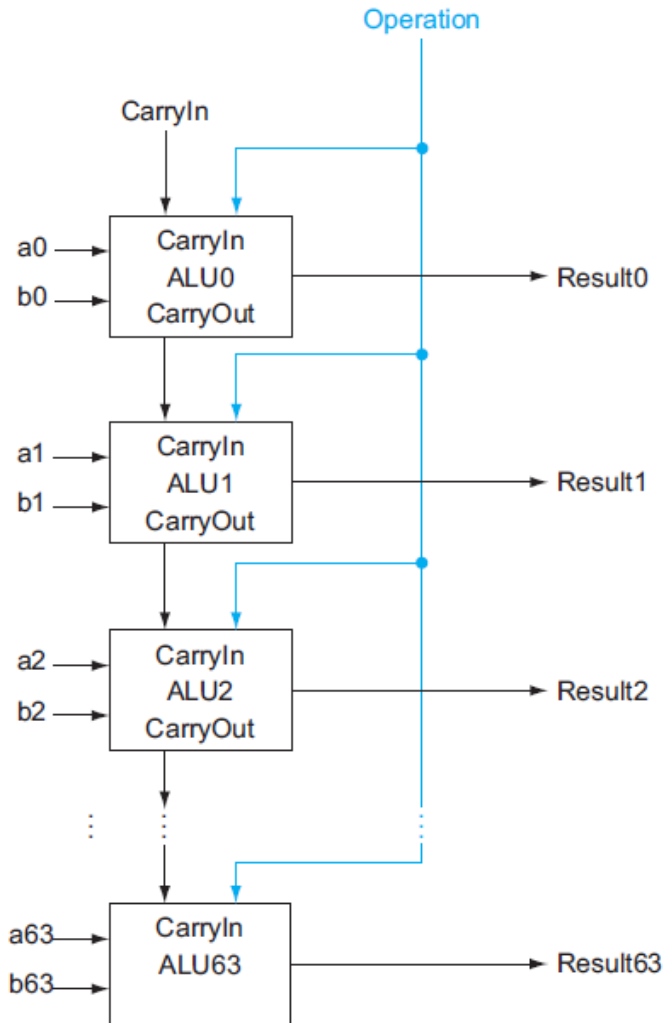


The 1-bit Logical Unit for AND and OR.



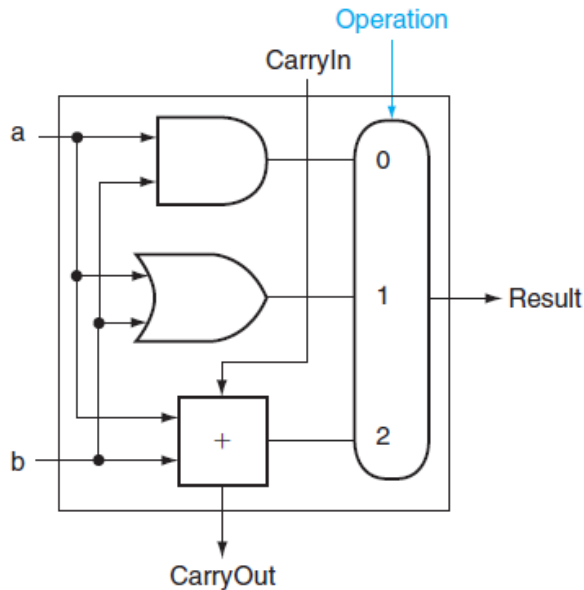
The 1-bit ALU that performs AND, OR, and addition.

Arithmetic Logic Unit (ALU)

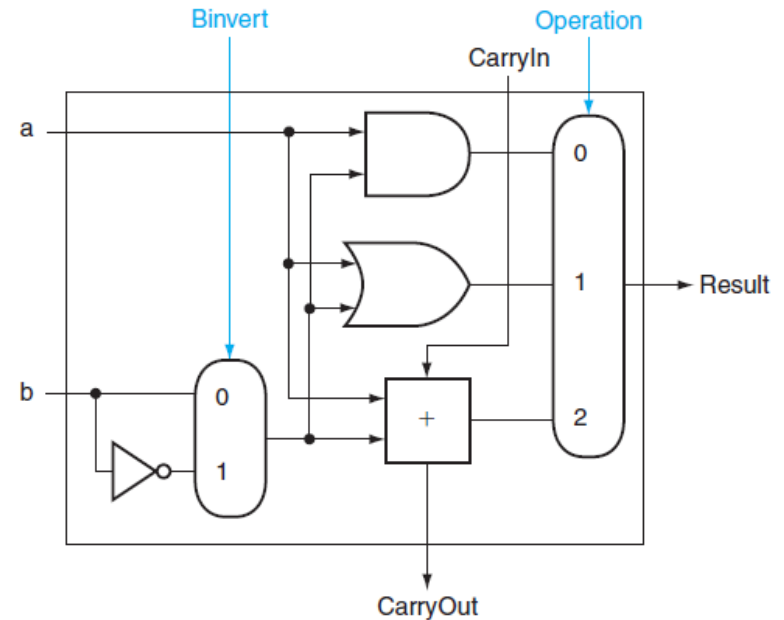


A 64-bit ALU constructed from 64 1-bit ALUs

Arithmetic Logic Unit (ALU)



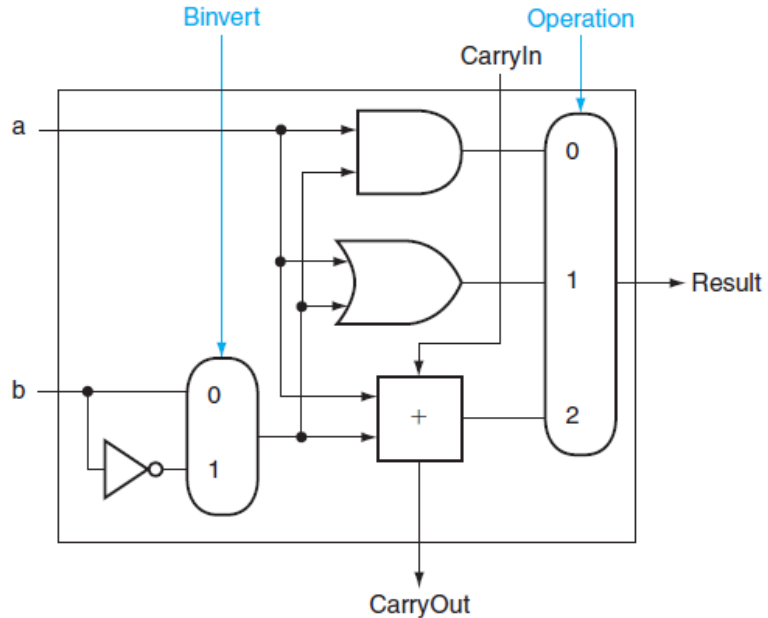
The 1-bit ALU that performs AND, OR, and addition.



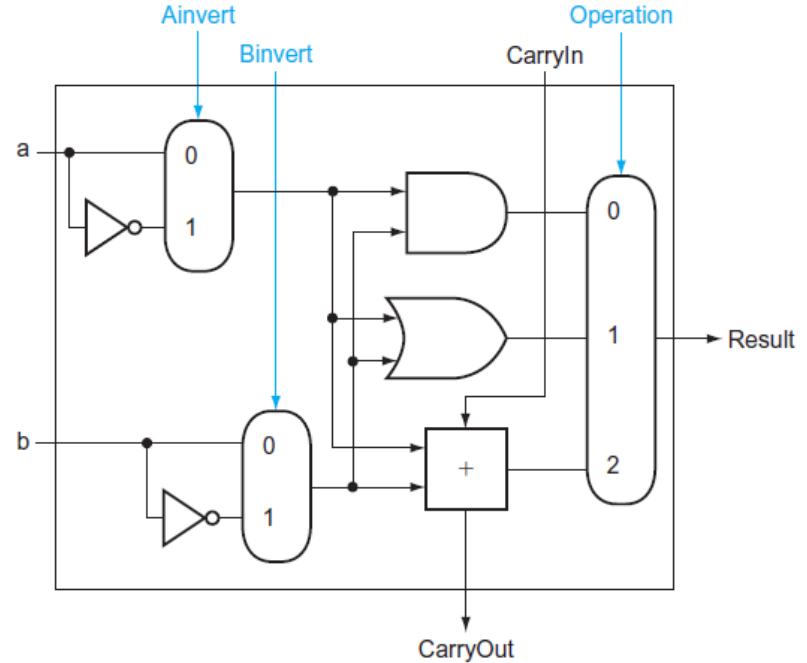
Support for subtraction (through addition).

- By selecting *b* (*Binvert* = 1) and setting *CarryIn* to 1 in the least significant bit of the ALU, we get two's complement subtraction of *b* from *a* instead of addition of *b* to *a*.

Arithmetic Logic Unit (ALU)



The 1-bit ALU that performs AND, OR, addition, and subtraction (through addition).

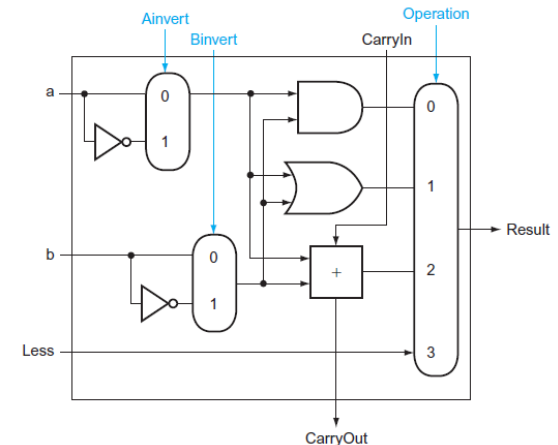
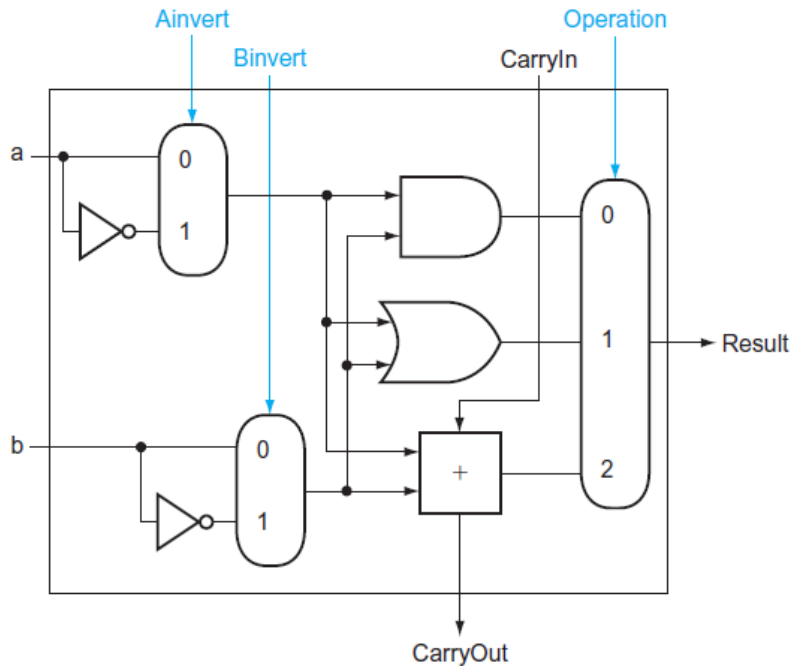


Support for NOR operation (through AND).

- DeMorgan's Theorem .

$$\overline{(a + b)} = \bar{a} \cdot \bar{b}$$

Arithmetic Logic Unit (ALU)

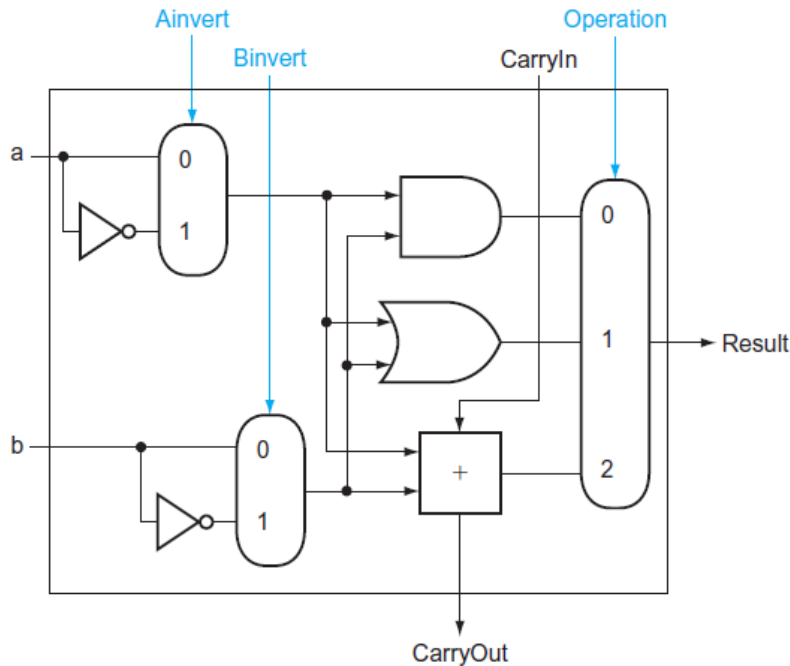


The 1-bit ALU that performs
AND, OR, addition, subtraction
(through addition) and NOR operation.

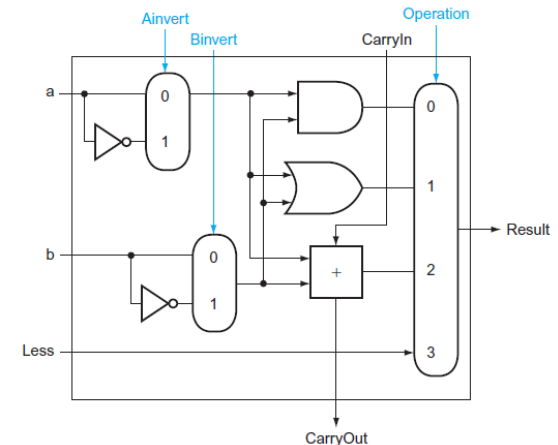
Support for slt operation.

- slt rd, rs1, rs2
rd = 1 if rs1 < rs2
rd = 0 otherwise
- Consequently, slt will set all but the least significant bit to 0, with the least significant bit set according to the comparison.

Arithmetic Logic Unit (ALU)



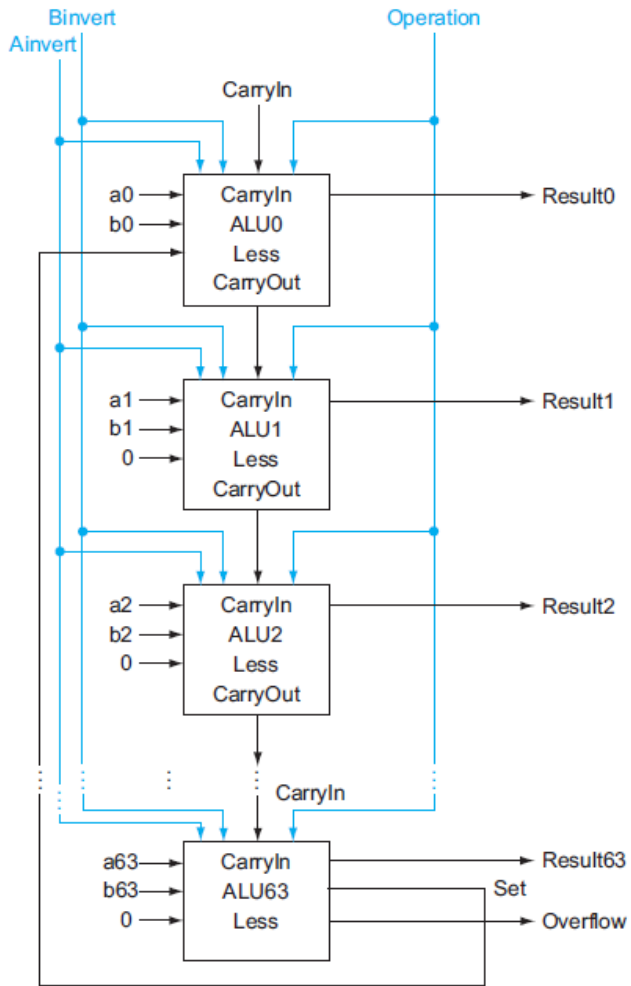
The 1-bit ALU that performs
AND, OR, addition, subtraction
(through addition) and NOR operation.



Support for slt operation.

- For the ALU to perform slt, we first need to expand the three-input multiplexor to add an input for the slt result. We call that new input Less and use it only for slt.

Arithmetic Logic Unit (ALU)

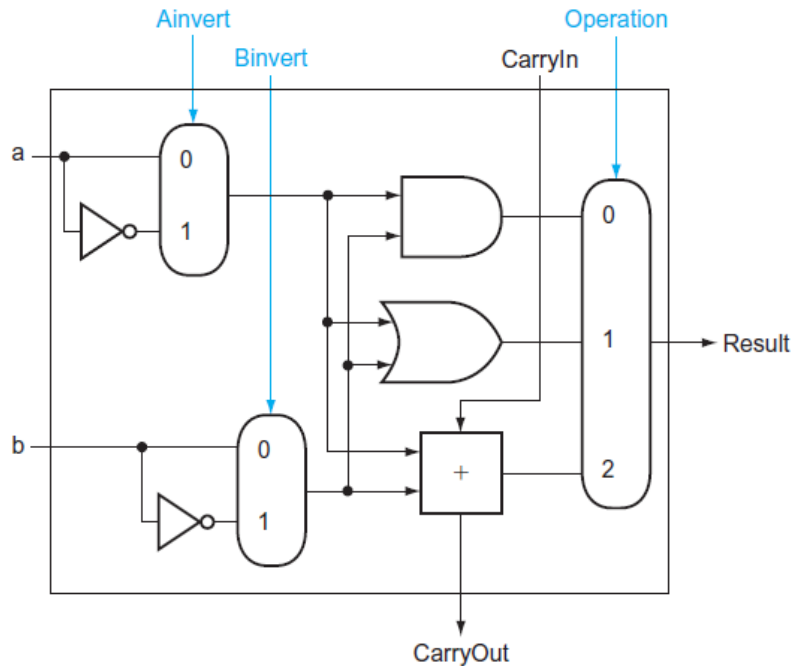


Support for slt operation.

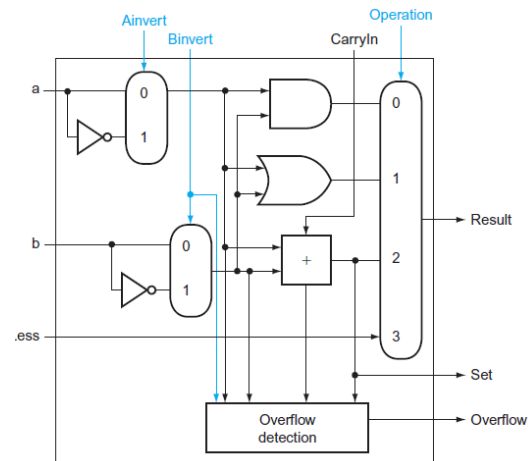
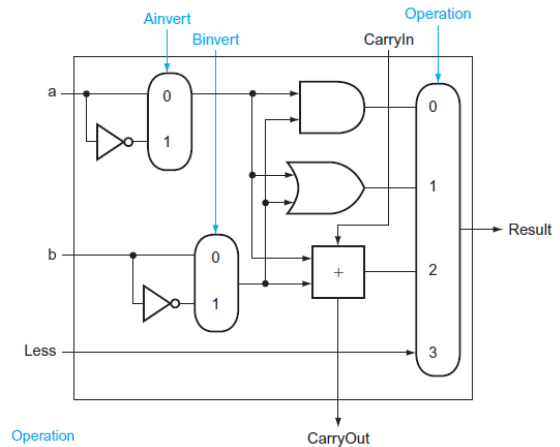
- We must connect 0 to the Less input for the upper 63 bits of the ALU, since those bits are always set to 0.
- How to set the least significant bit?
 - We want the least significant bit of slt operation to be a 1 if $a < b$
 - That is, we want the least significant bit of slt operation to be a 1 if $a - b$ is negative and a 0 if $a - b$ is positive
 - This desired result corresponds exactly to the sign bit values after performing $a - b$: 1 means negative and 0 means positive
 - So, we need only connect the sign bit (adder output of the most significant bit) to the Less input of least significant bit to get slt.
 - This needs a modified 1-bit ALU at the most significant bit

A 64-bit ALU constructed from 64 1-bit ALUs (Notice the ALU for Most Significant Bit)

Arithmetic Logic Unit (ALU)



The 1-bit ALU that performs AND, OR, addition, subtraction (through addition) and NOR operation.

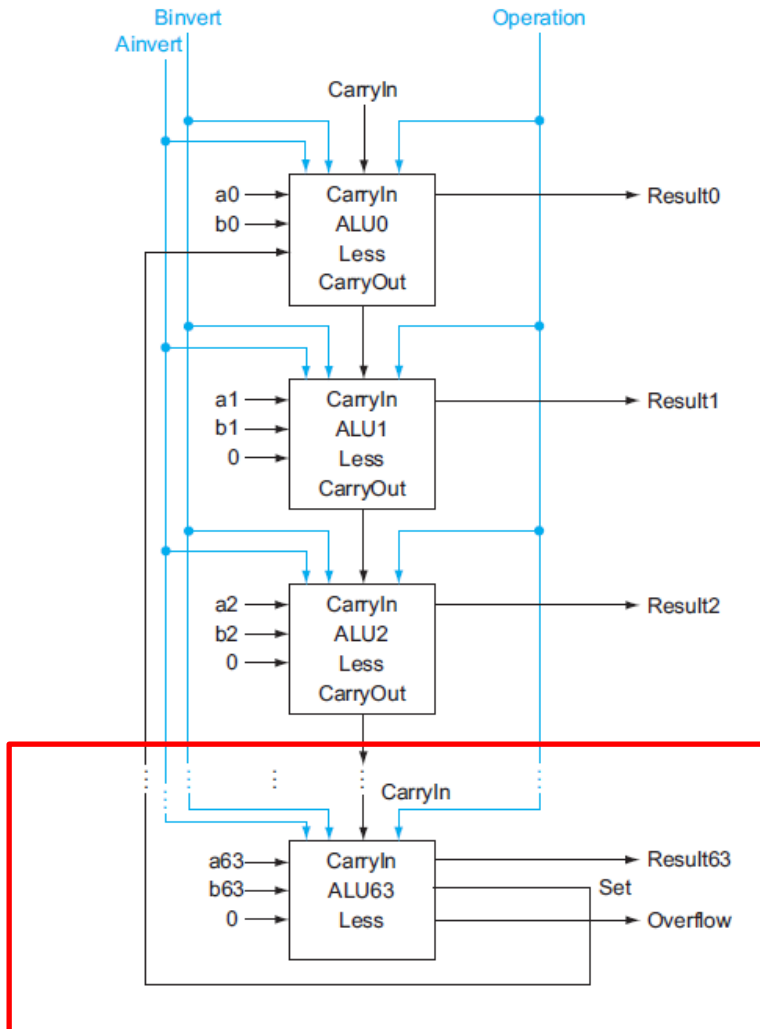


← Most significant bit

Support for slt operation.

- We need to connect the sign bit (adder output of the most significant bit) to the Less input of least significant bit to get slt.
- This needs a modified 1-bit ALU at the most significant bit (addition of the Set output)

Arithmetic Logic Unit (ALU)

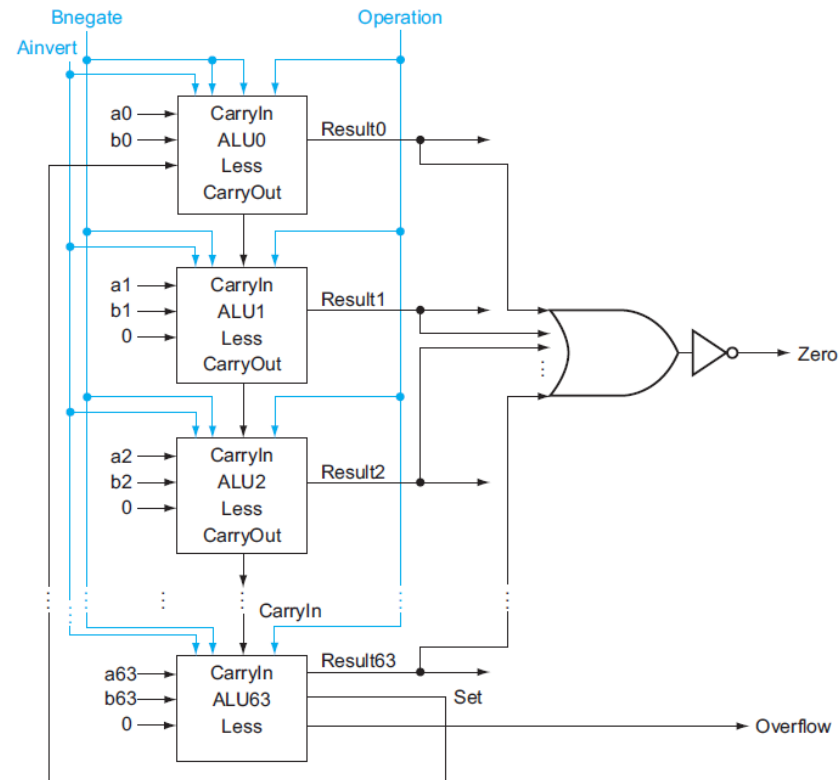


Support for slt operation.

- We must connect 0 to the Less input for the upper 63 bits of the ALU, since those bits are always set to 0.
- How to set the least significant bit?
 - We want the least significant bit of slt operation to be a 1 if $a < b$
 - That is, we want the least significant bit of slt operation to be a 1 if $a - b$ is negative and a 0 if $a - b$ is positive
 - This desired result corresponds exactly to the sign bit values after performing $a - b$: 1 means negative and 0 means positive
 - So, we need only connect the sign bit (adder output of the most significant bit) to the Less input of least significant bit to get slt.
 - This needs a modified 1-bit ALU at the most significant bit

A 64-bit ALU constructed from 64 1-bit ALUs (Notice the ALU for Most Significant Bit)

Arithmetic Logic Unit (ALU)

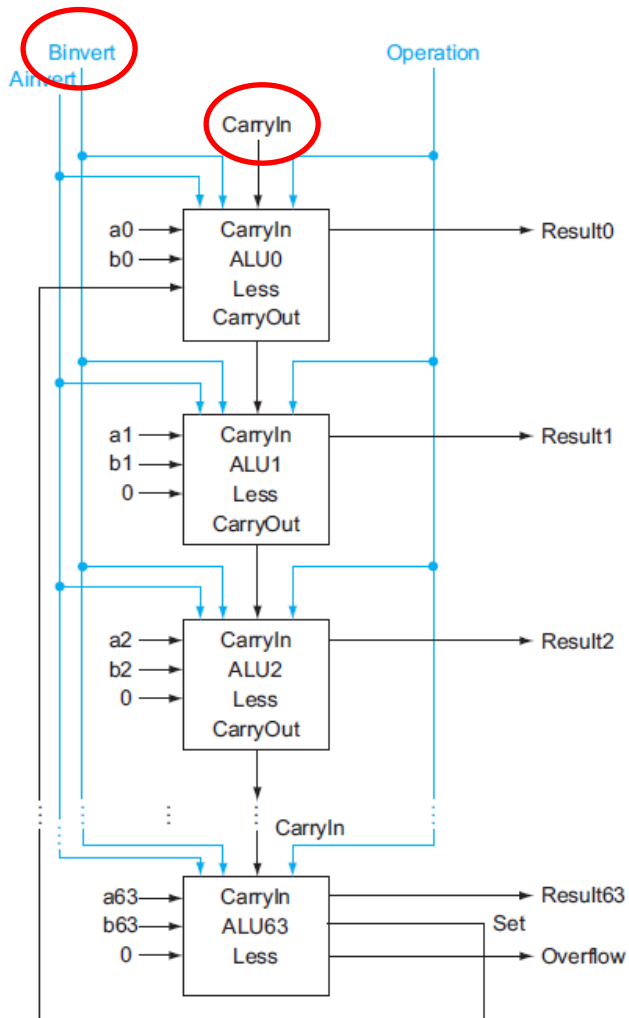


Support for beq operation.

- beq operation branches if two registers are equal.
- The easiest way to test equality with the ALU is to subtract b from a and then test to see if the result is 0.
- To test if the result is 0, the simplest way is to OR all the outputs together and then send that signal through an inverter

$$\text{Zero} = \overline{(\text{Result63} + \text{Result62} + \dots + \text{Result2} + \text{Result1} + \text{Result0})}$$

Arithmetic Logic Unit (ALU)



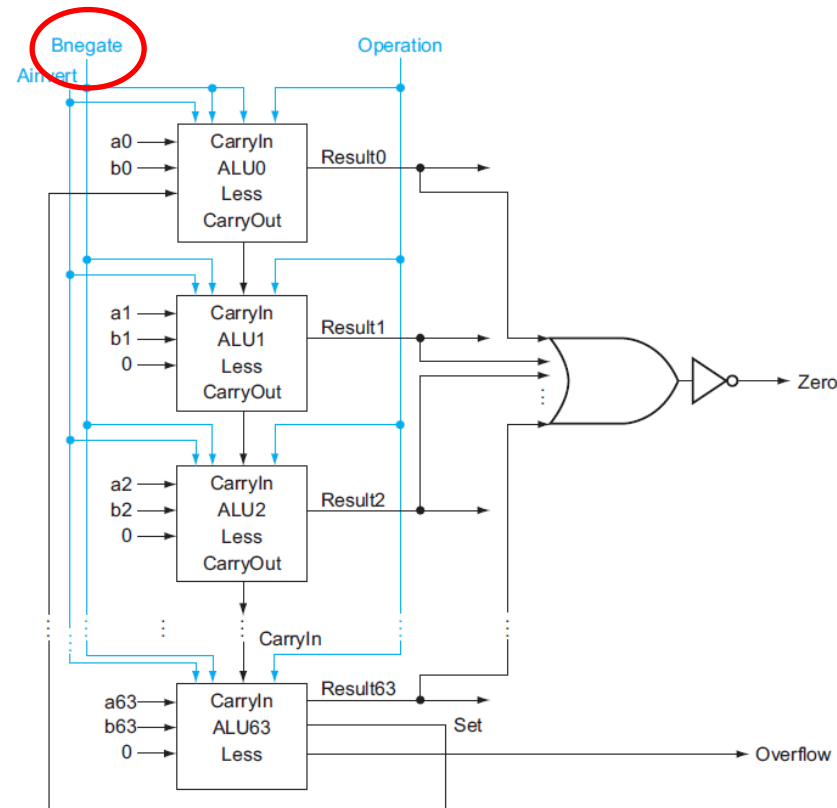
Simplifying Control

- Notice that every time we want the ALU to subtract, we set both CarryIn and Binvert to 1.
- For adds or logical operations, we want both control lines to be 0.
- We can therefore simplify control of the ALU by combining the CarryIn and Binvert to a single control line called Bnegate.

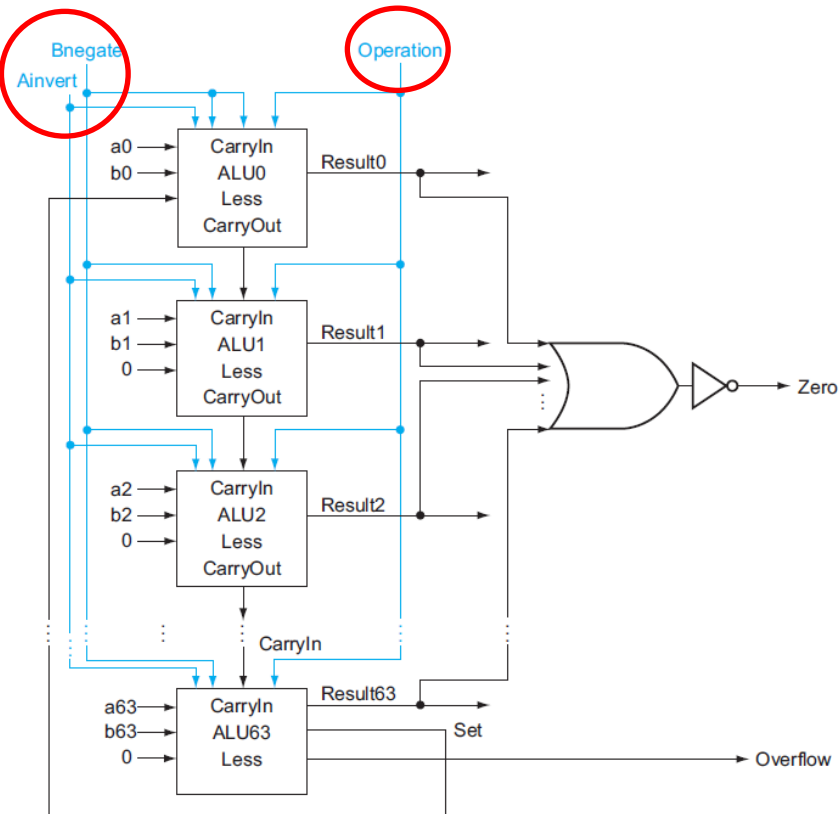
Arithmetic Logic Unit (ALU)

Simplifying Control

- Notice that every time we want the ALU to subtract, we set both CarryIn and Binvert to 1.
- For adds or logical operations, we want both control lines to be 0.
- We can therefore simplify control of the ALU by combining the CarryIn and Binvert to a single control line called Bnegate.



Arithmetic Logic Unit (ALU)

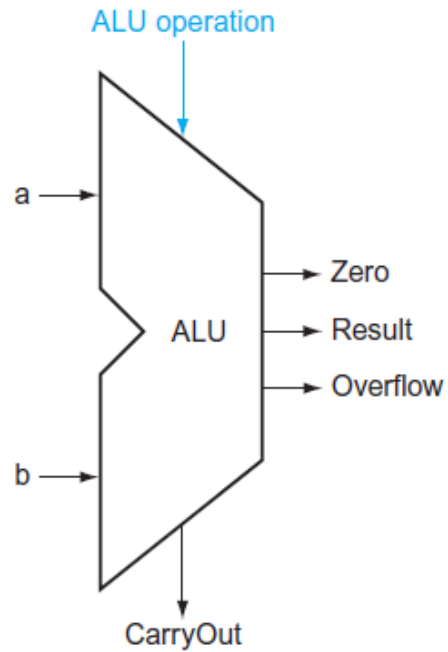


ALU Control

- We can think of the combination of the 1-bit Ainvert line, the 1-bit Bnegate line, and the 2-bit Operation lines as **4-bit control lines for the ALU**, telling it to perform add, subtract, AND, OR, NOR, or set less than.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

Arithmetic Logic Unit (ALU)



Symbol for a Complete ALU

Processor Implementation (Chapter 4)

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

Clock cycle time = Clock Period = Time period of the clock signal

- Instruction Count
 - Determined by Compiler and Instruction Set Architecture
- CPI and Clock cycle time
 - Determined by Processor Implementation

Processor Implementation (Chapter 4)

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

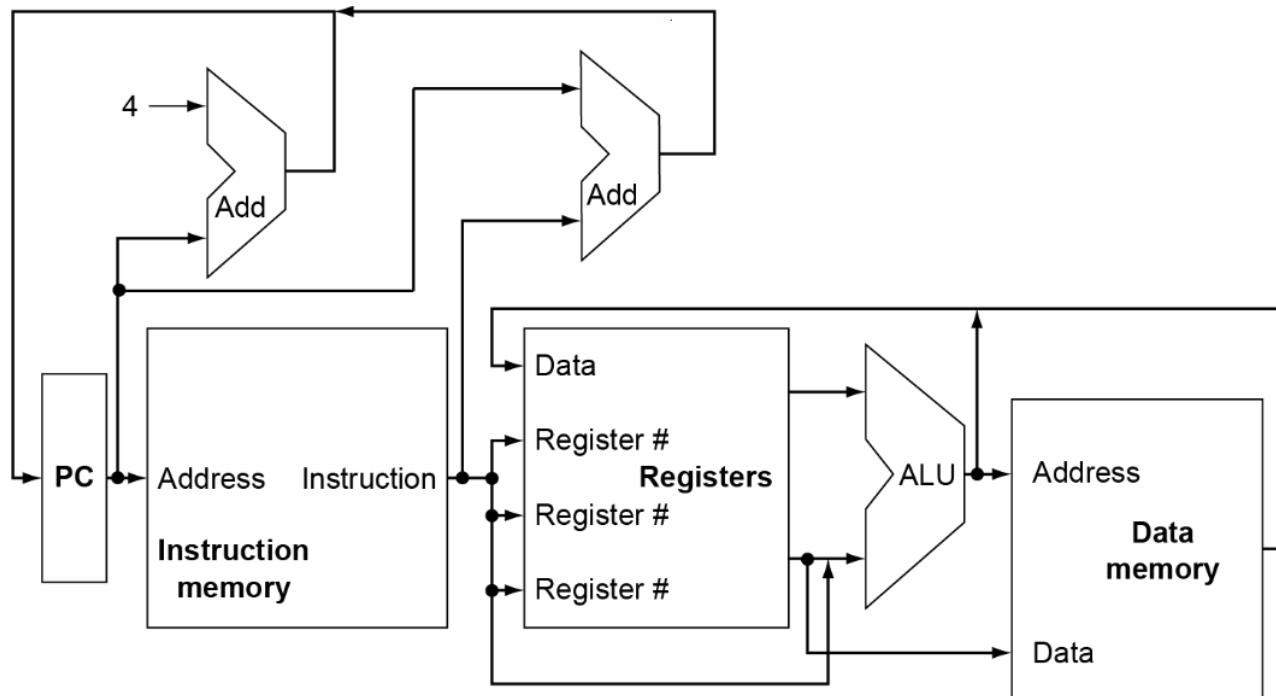
Clock cycle time = Clock Period = Time period of the clock signal

- Instruction Count
 - Determined by Compiler and Instruction Set Architecture
- CPI and Clock cycle time
 - Determined by Processor Implementation
 - We will examine two RISC-V processor implementations
 - A simplified version
 - A more realistic pipelined version

RISC-V Processor Implementation: Simplified Version

- Supports a simplified subset of RISC-V instructions
 - Memory reference: **ld, sd**
 - Arithmetic/logical: **add, sub, and, or**
 - Control transfer: **beq**

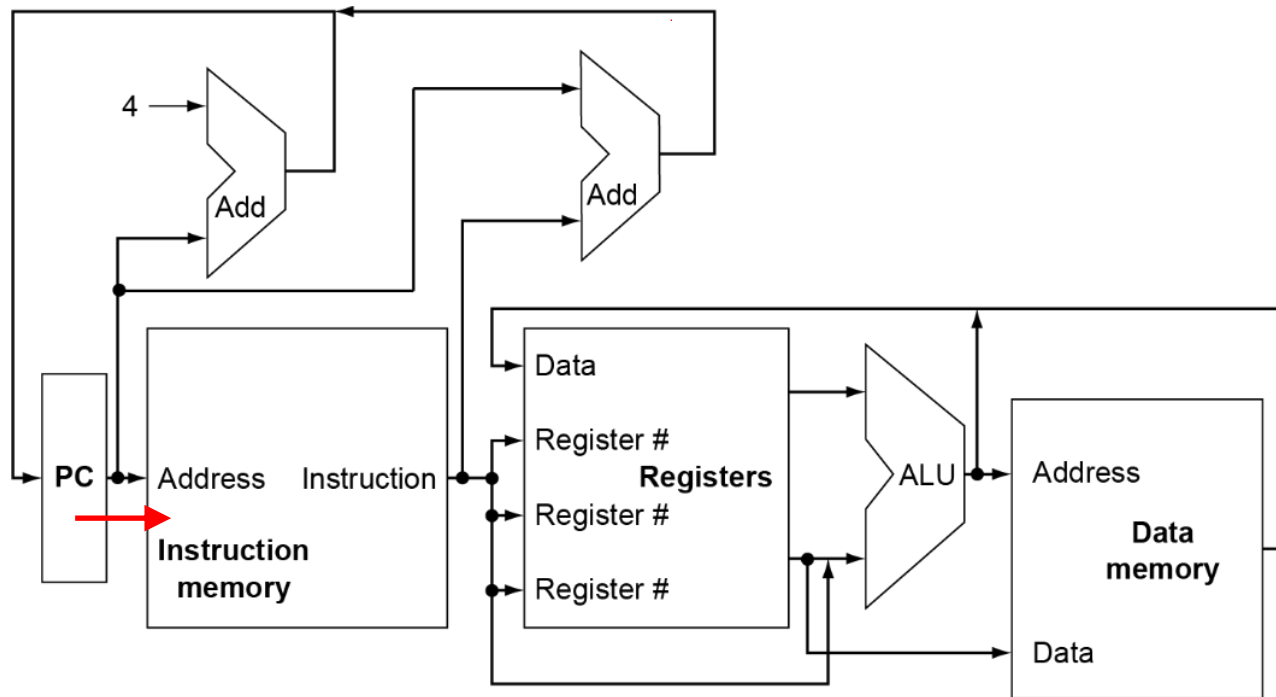
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction

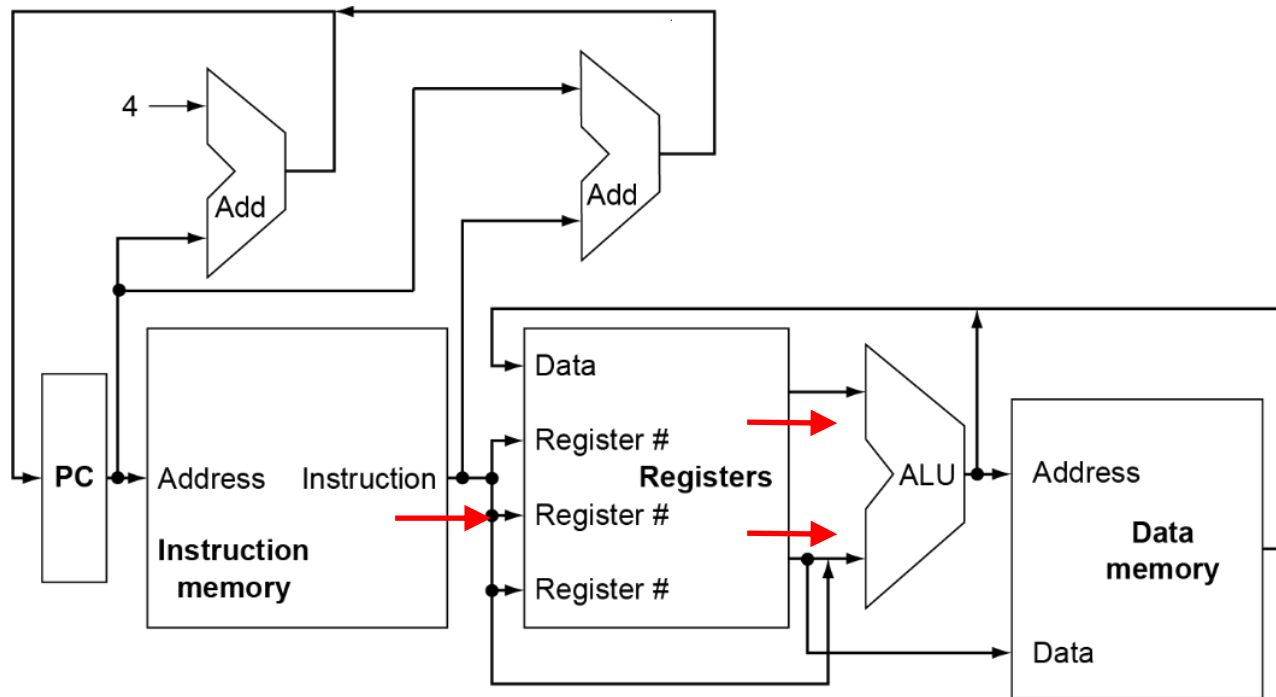
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers

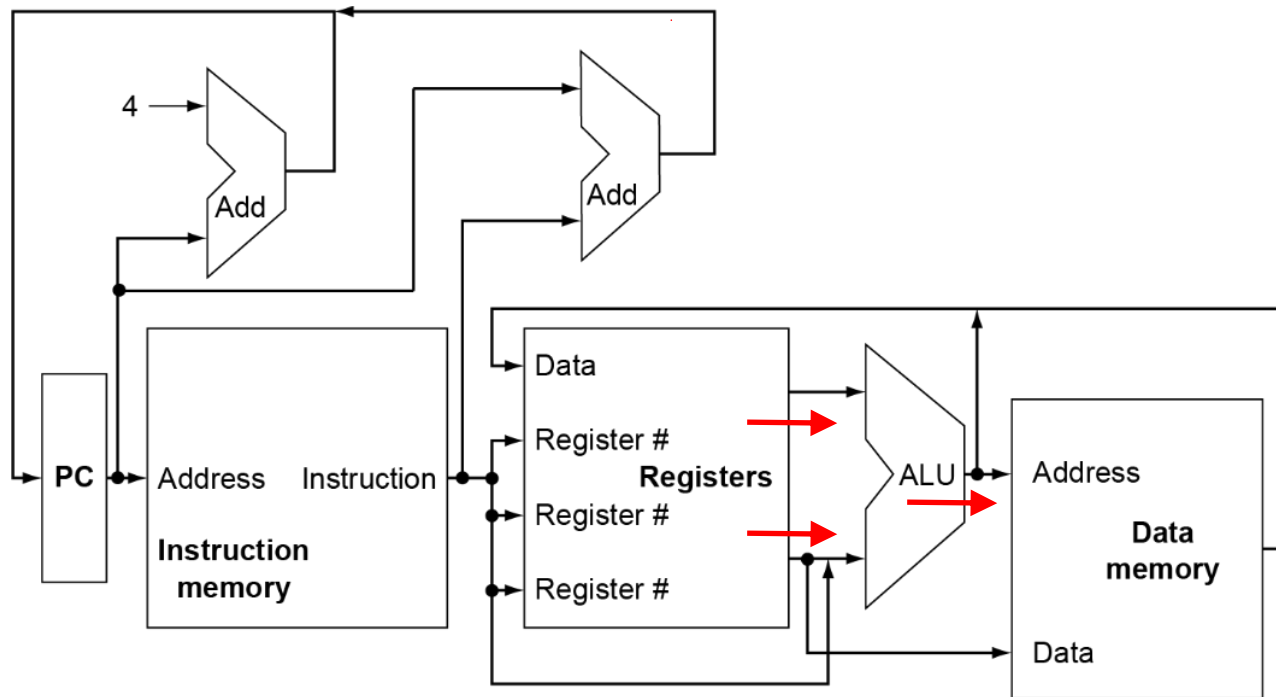
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers
3. Depending on instruction class, use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison

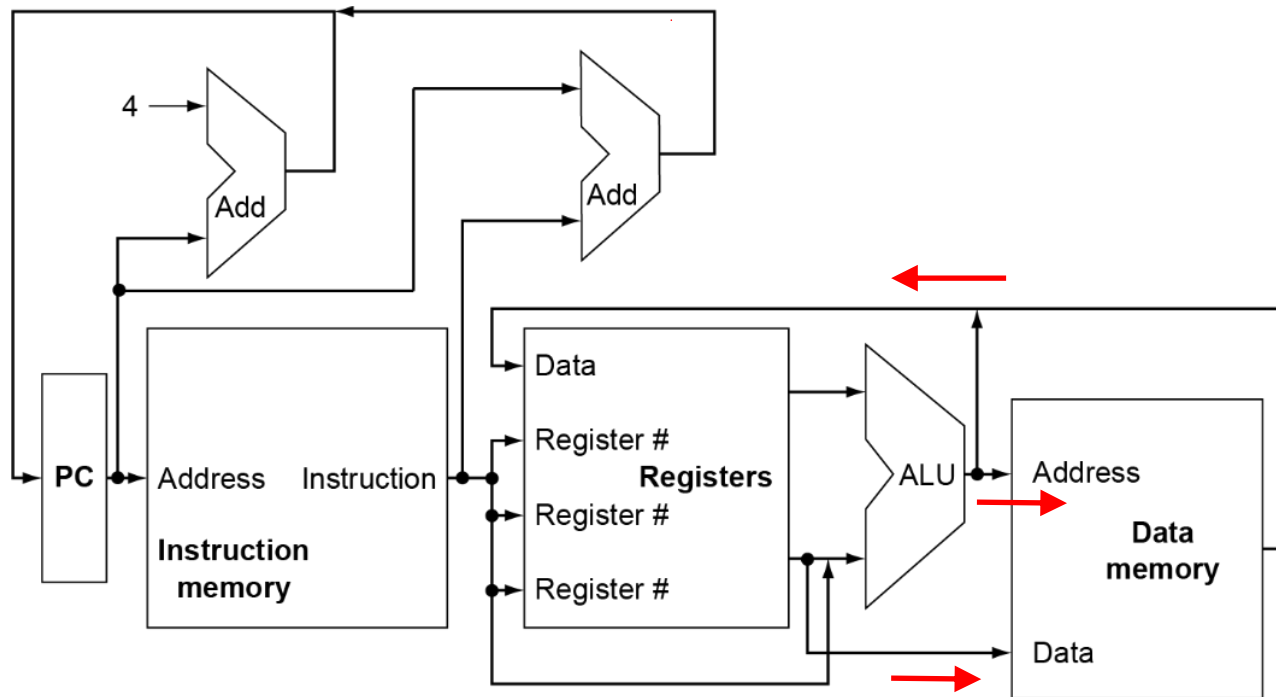
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers
3. Depending on instruction class, use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison
4. Access data memory for load/store

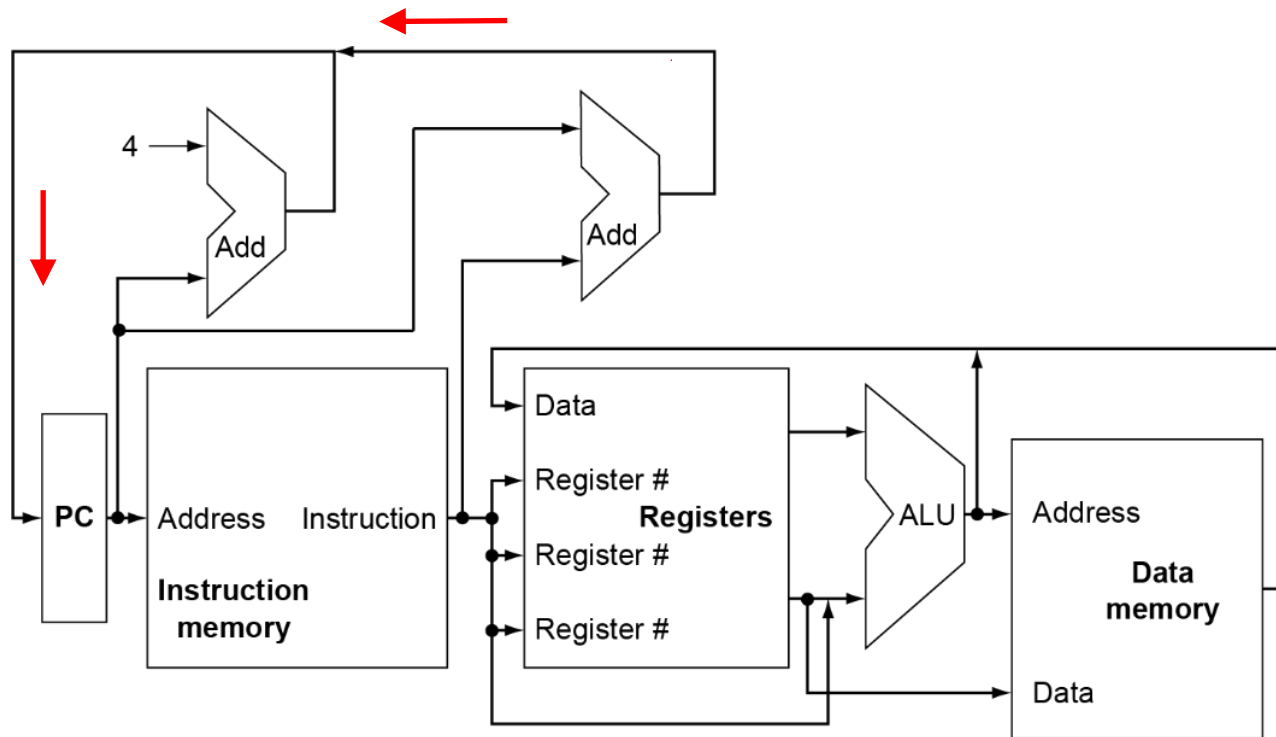
RISC-V Processor Implementation: Overview



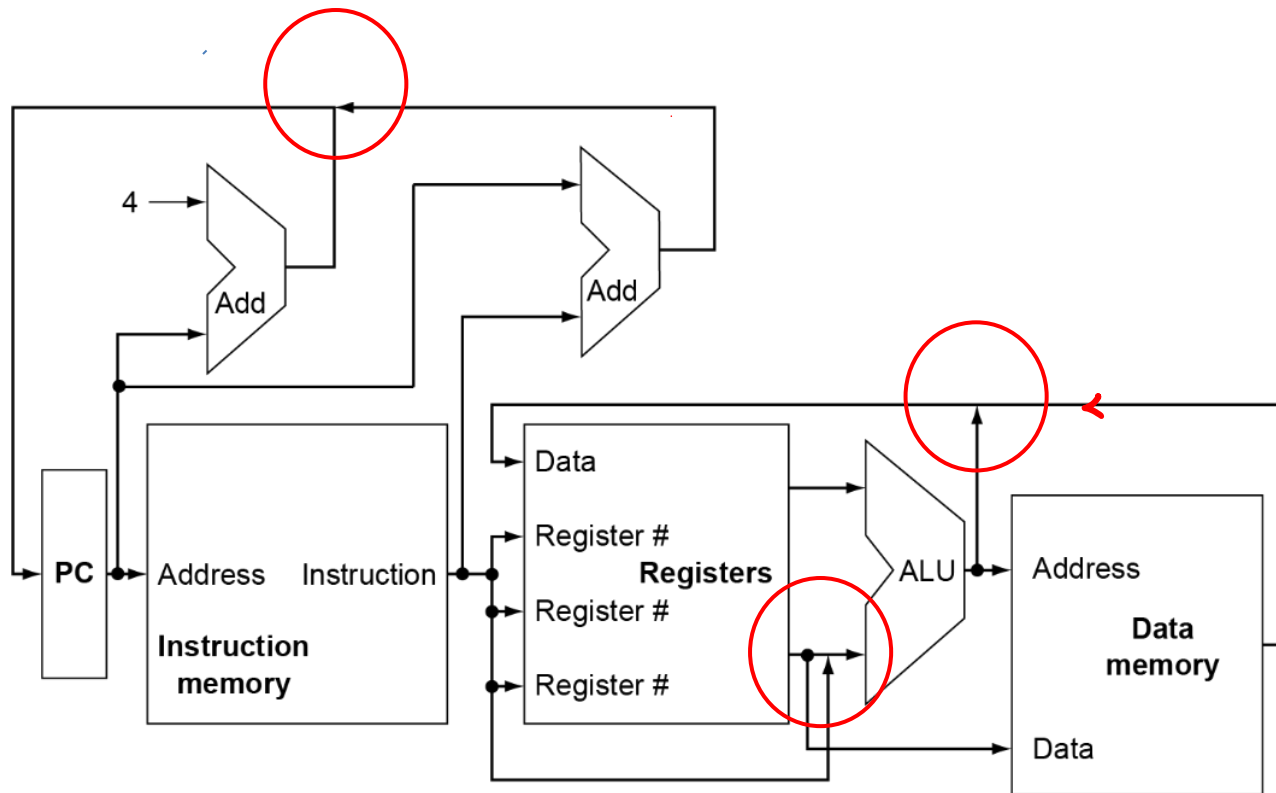
RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers
3. Depending on instruction class, use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison
4. Access data memory for load/store
5. PC → target address or PC + 4

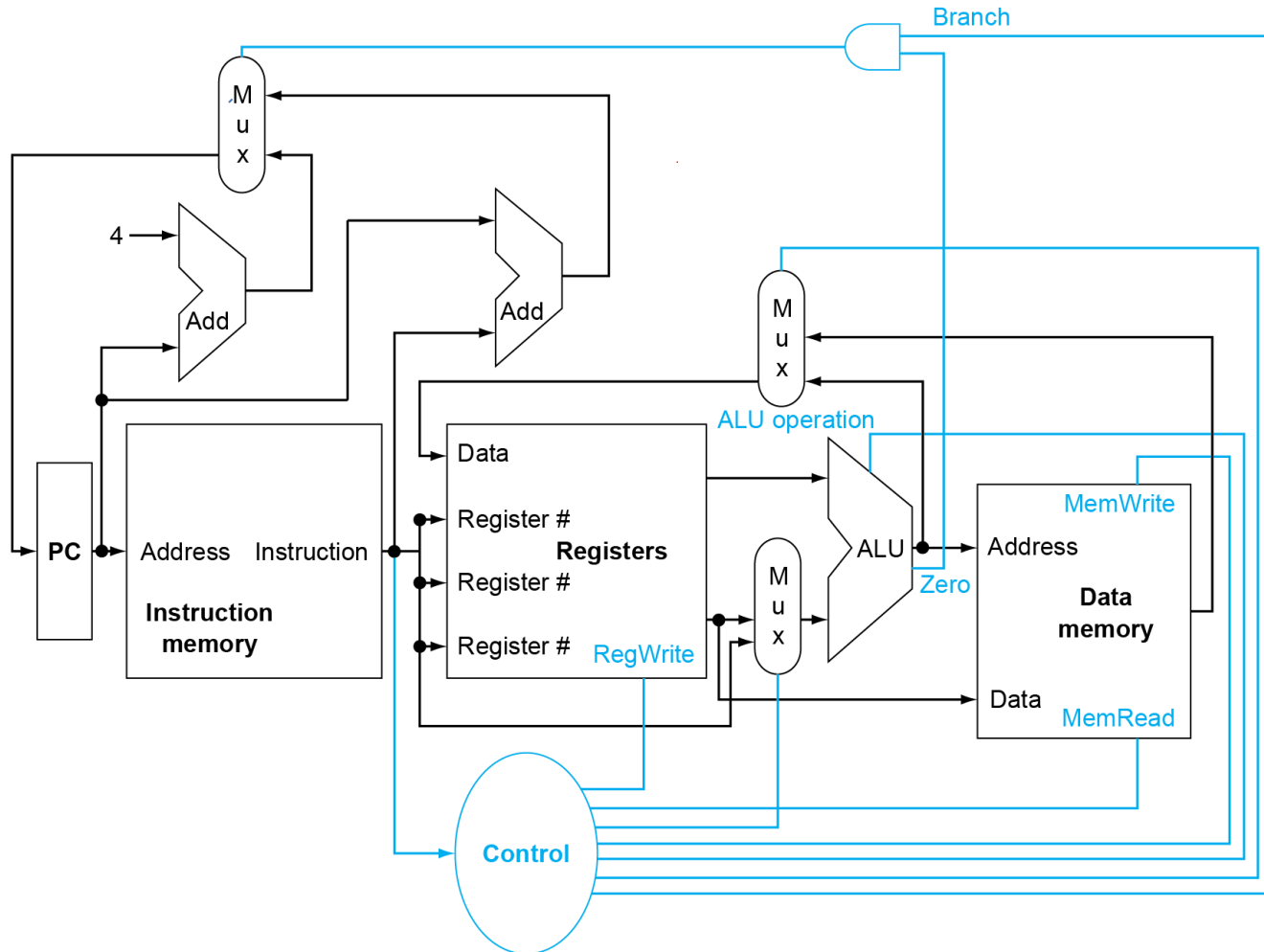
RISC-V Processor Implementation: Overview



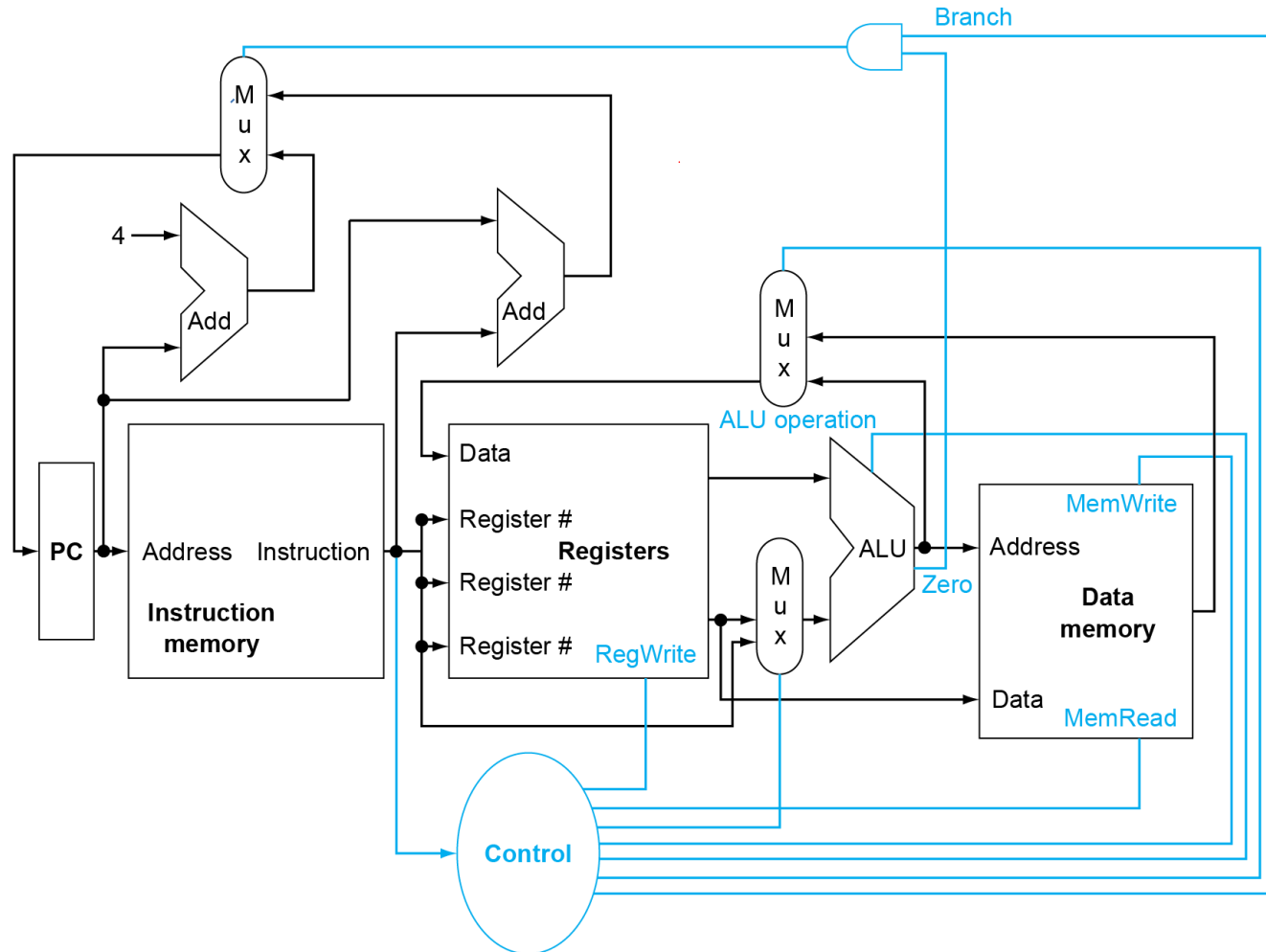
RISC-V Processor Implementation: Multiplexers



RISC-V Processor Implementation: Control



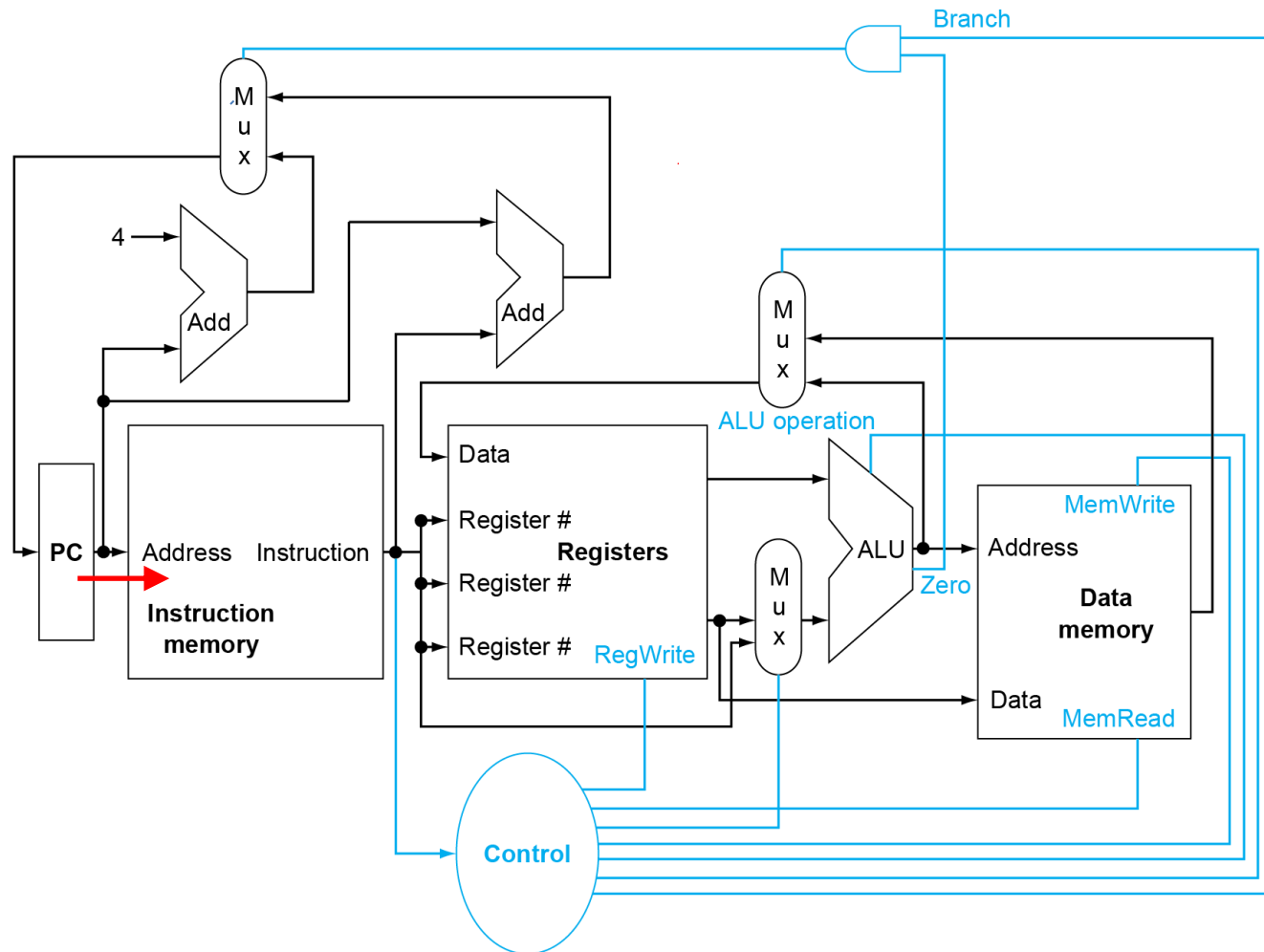
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction

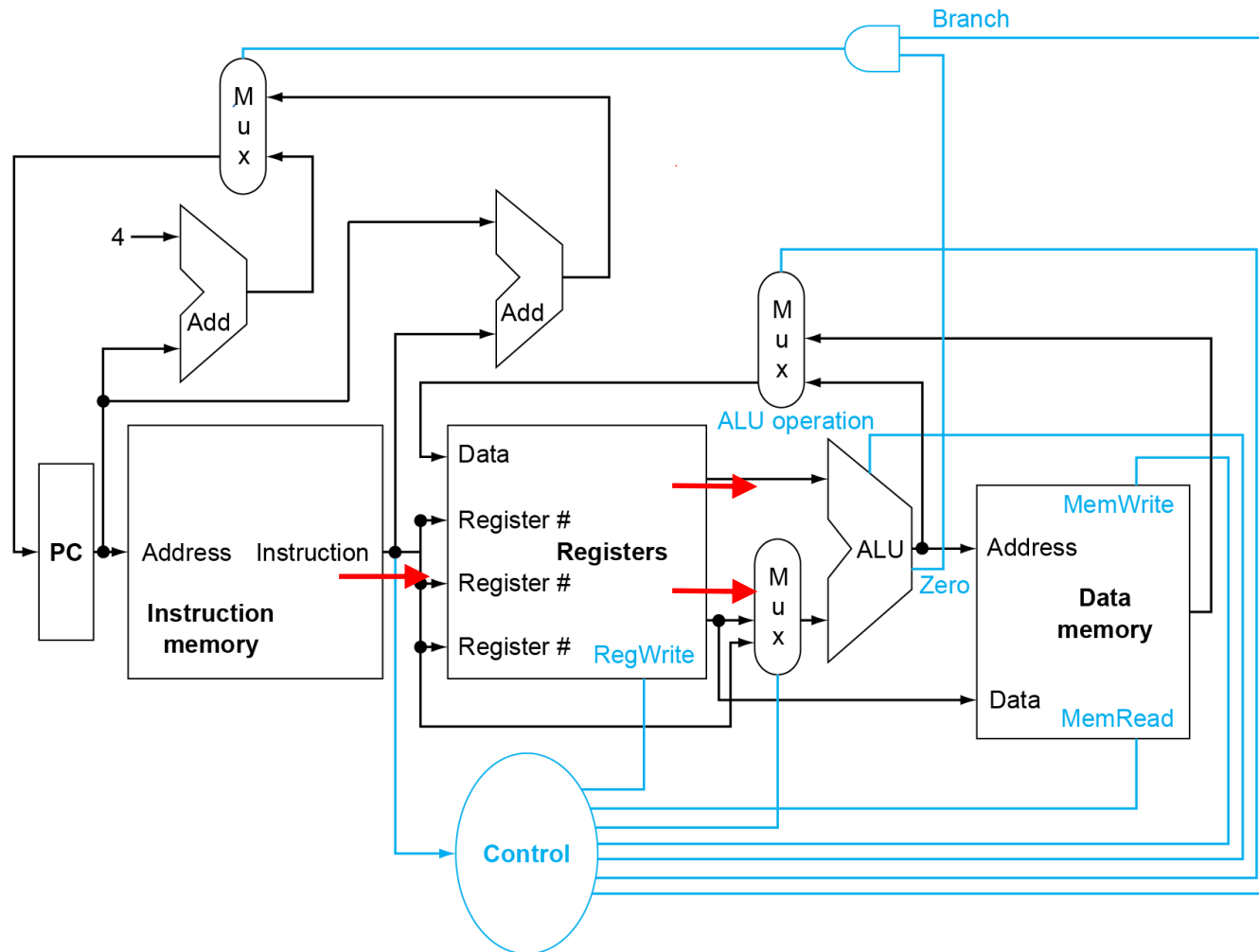
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers

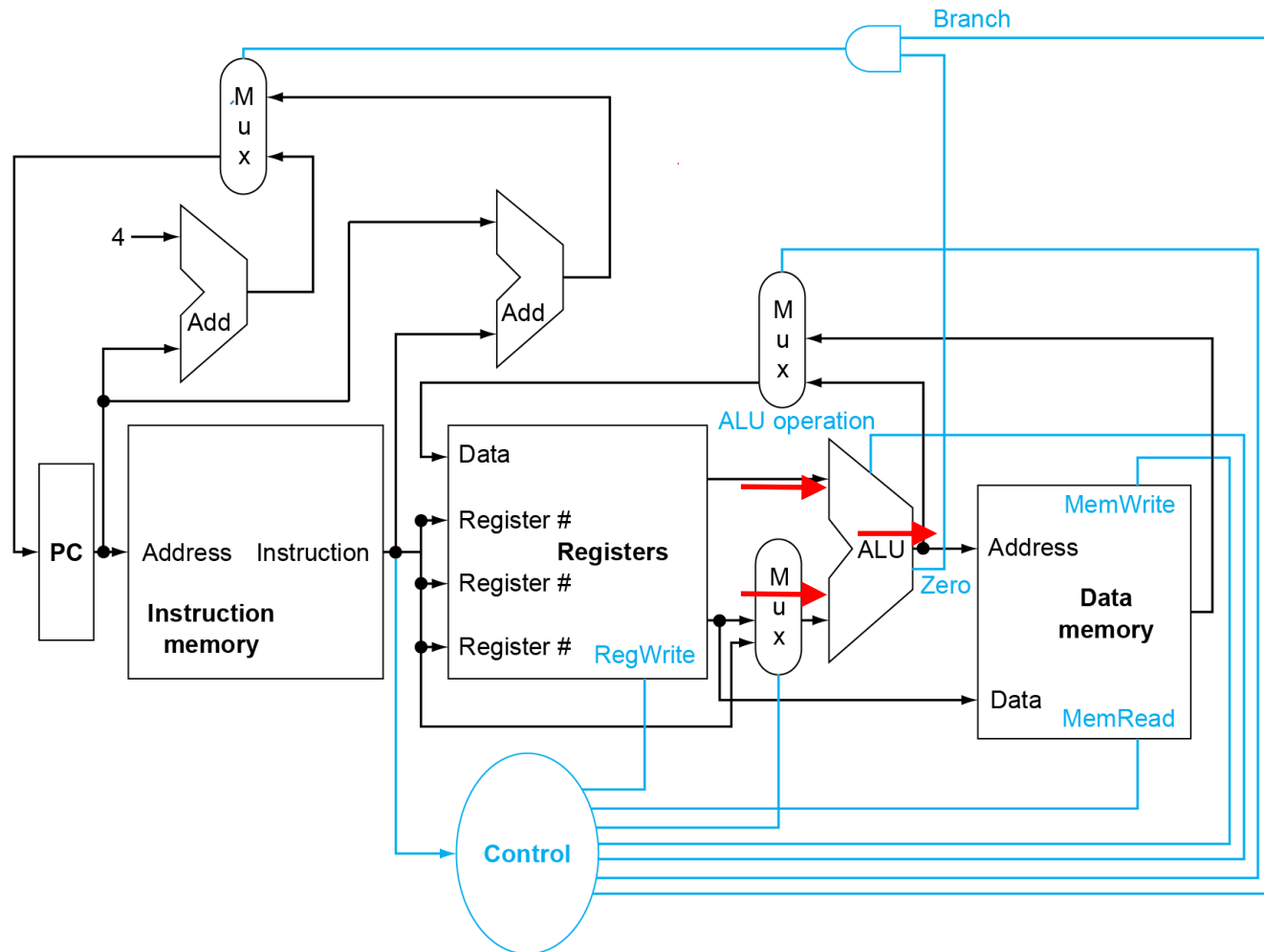
RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers
3. Depending on instruction class, use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison

RISC-V Processor Implementation: Overview



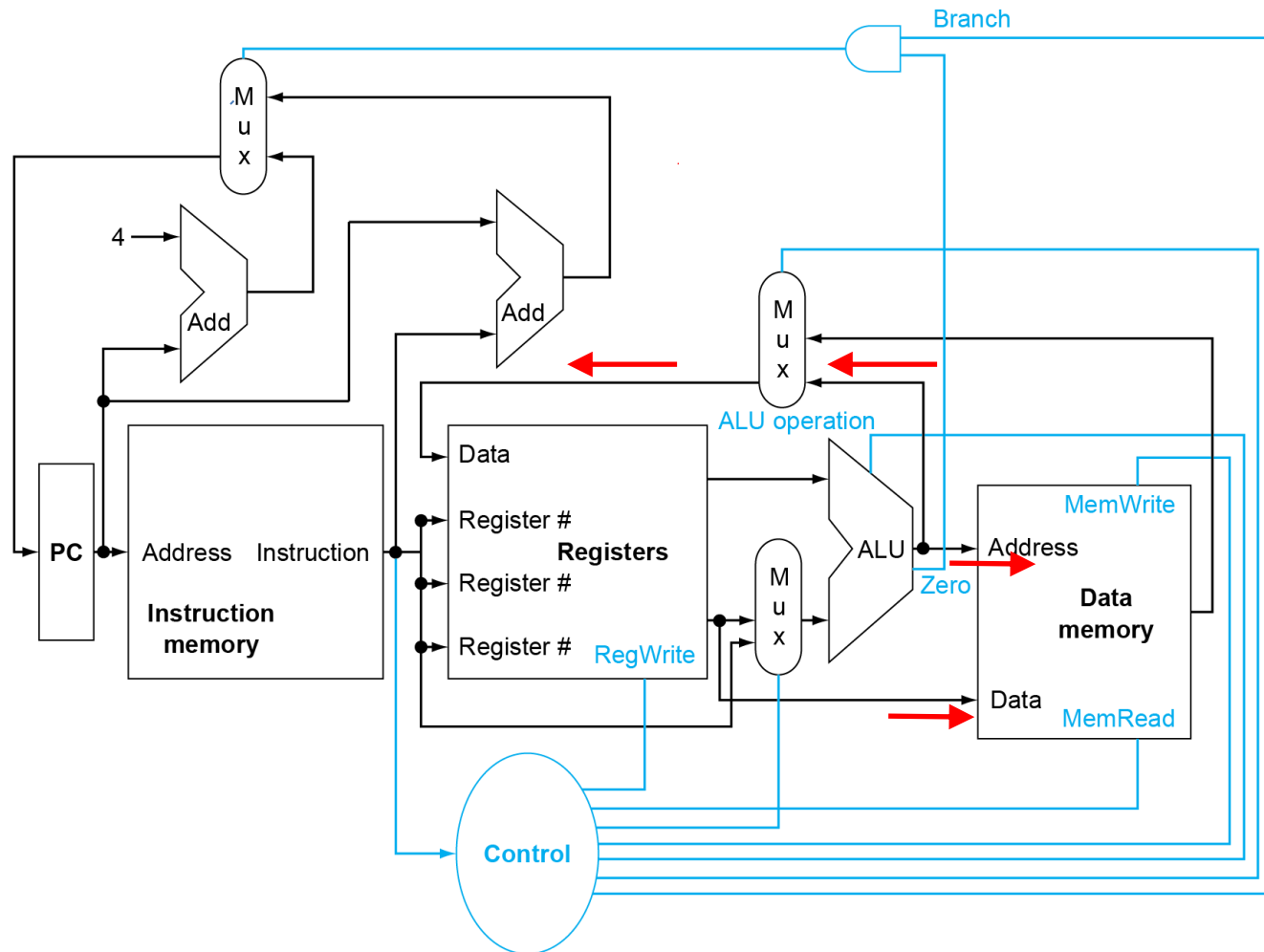
RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers
3. Depending on instruction class, use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison
4. Access data memory for load/store

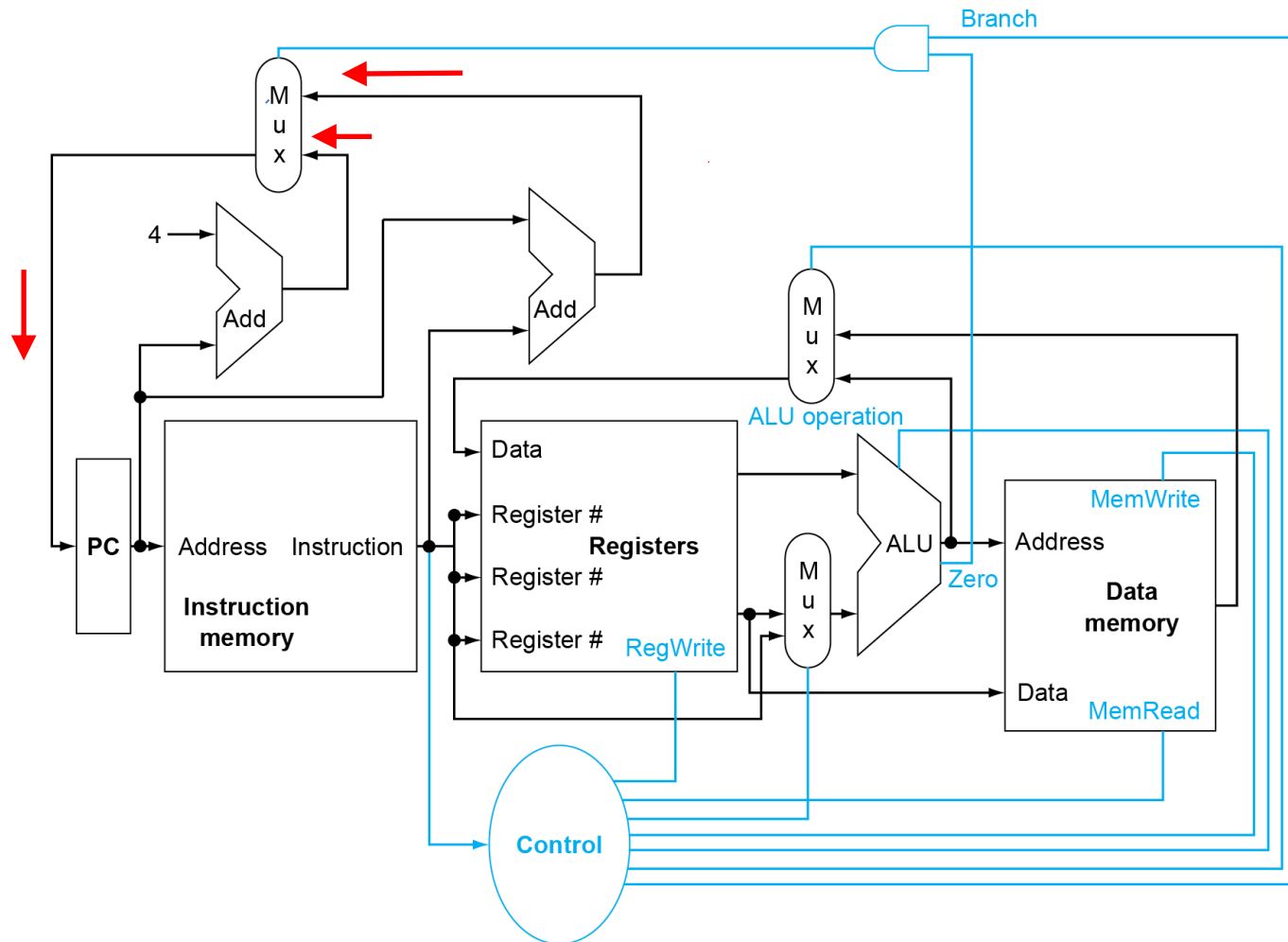
RISC-V Processor Implementation: Instruction Execution

1. Program Counter (PC) → Instruction memory
 - fetch instruction
2. Register numbers → register file
 - read registers
3. Depending on instruction class, use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison
4. Access data memory for load/store
5. PC → target address or PC + 4

RISC-V Processor Implementation: Overview



RISC-V Processor Implementation: Overview



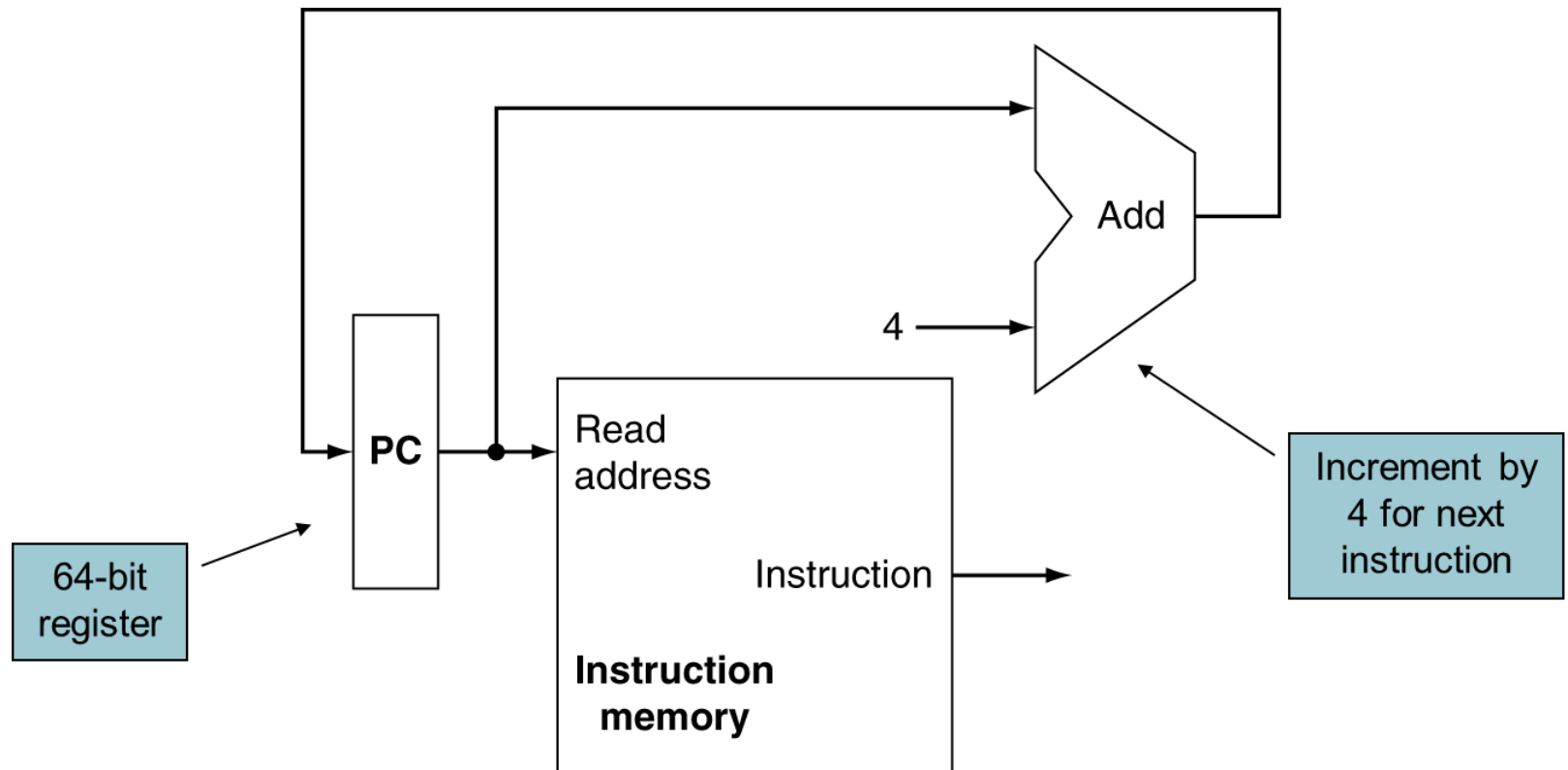
RISC-V Processor Implementation: Overview

- **Datapath Design**
- Control Lines and Control Unit

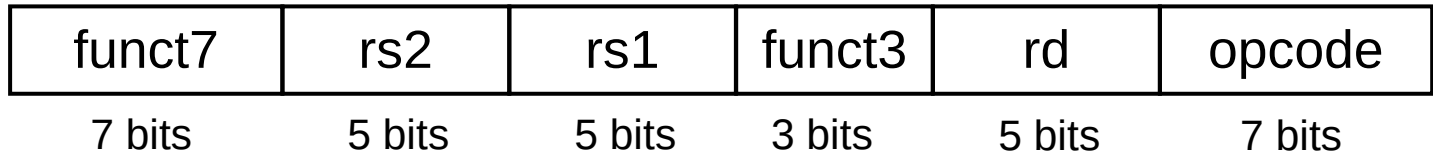
Building a Datapath

- Datapath
 - Elements that process data and addresses in the CPU
 - Examples: Registers, ALUs, mux's, memories, ...
- We will build a RISC-V datapath incrementally

Building a Datapath: Instruction Fetch



RISC-V R-format Instructions

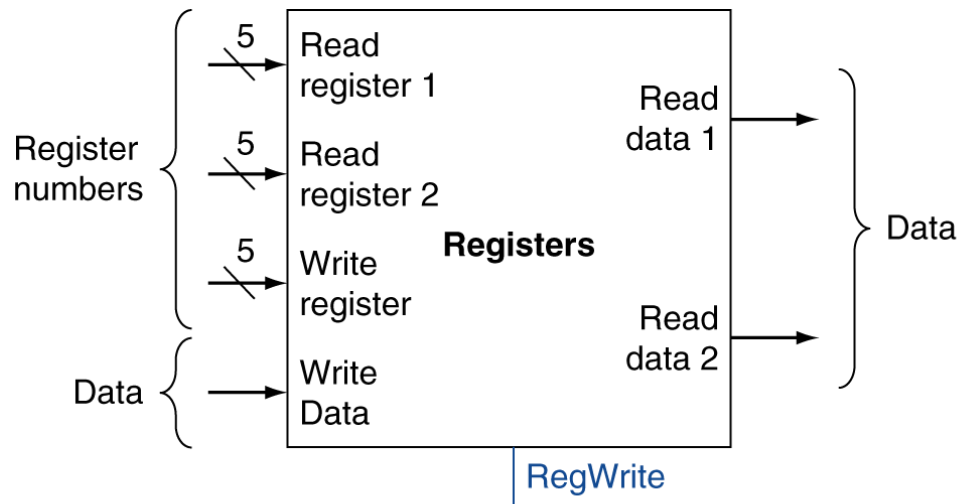


- Instruction fields
 - opcode: operation code
 - rd: destination register number
 - funct3: 3-bit function code (additional opcode)
 - rs1: the first source register number
 - rs2: the second source register number
 - funct7: 7-bit function code (additional opcode)

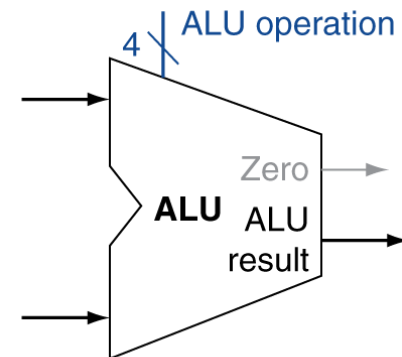
```
add x9, x20, x21
```

Building a Datapath: R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

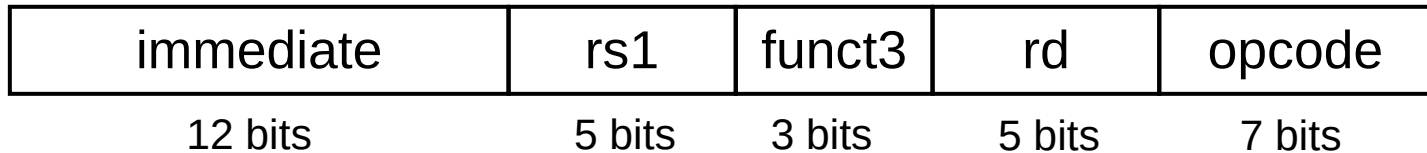


a. Registers



b. ALU

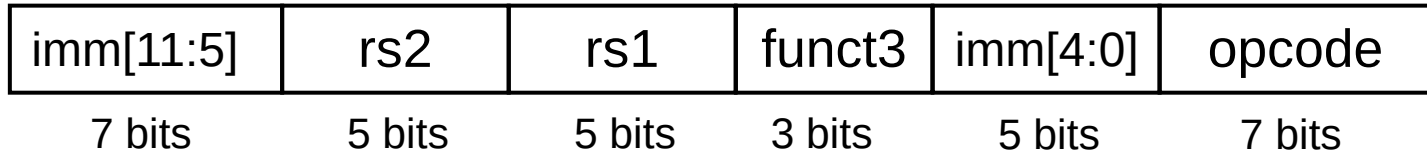
RISC-V I-format Load Instruction



- Instruction fields
 - opcode: operation code
 - rd: destination register number
 - funct3: 3-bit function code (additional opcode)
 - rs1: source or base address register number
 - Immediate: constant operand, or offset added to base address (2's complement, sign extended)

```
ld x9, 64(x22)
```


RISC-V S-format Store Instructions

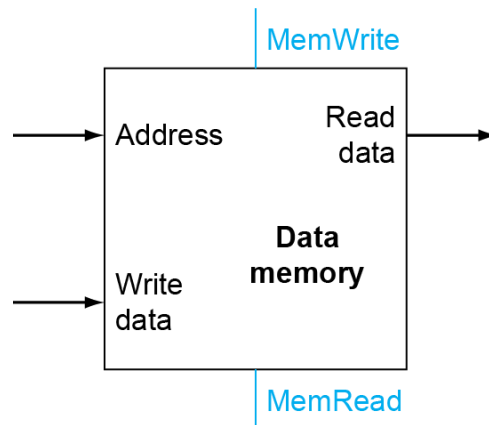


- Instruction fields
 - opcode: operation code
 - funct3: 3-bit function code (additional opcode)
 - rs1: base address register number
 - rs2: source operand register number
 - immediate: offset added to base address (Split so that rs1 and rs2 fields always in the same place across various instruction formats)

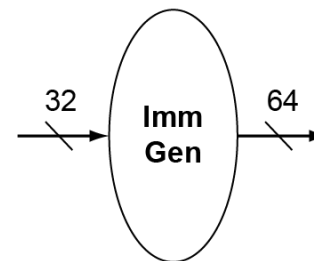
```
sd    x9, 240(x10)
```

Building a Datapath: Load-Store Instructions

- Read register operands
- Calculate address using 12-bit offset
 - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



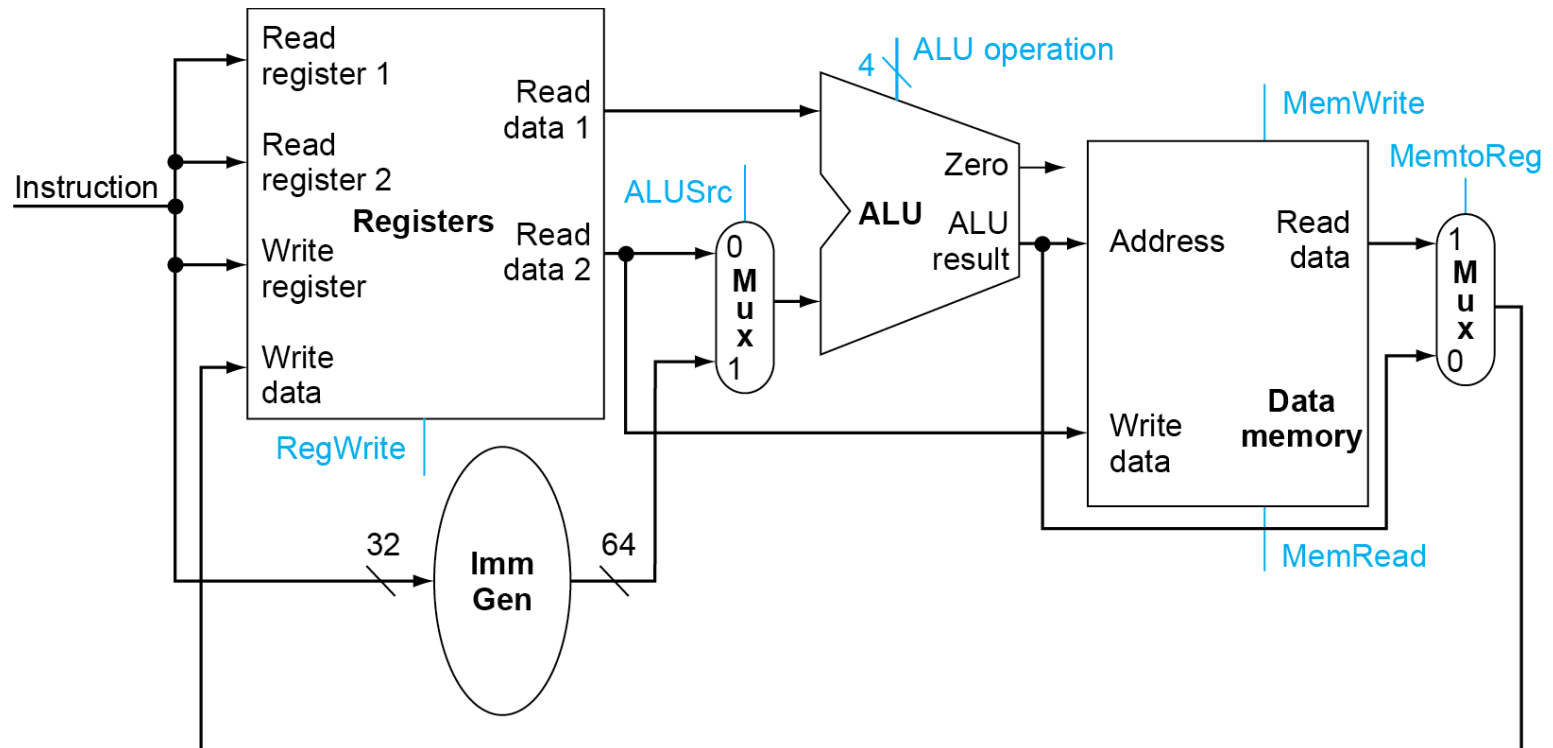
a. Data memory unit



b. Immediate generation unit

Building a Datapath: Composing the Elements

- R-Type + Load-Store Datapath



Conditional Branches

- Branch to a labeled instruction if a condition is true
 - Otherwise, continue sequentially
- **beq rs1, rs2, L1**
 - if (rs1 == rs2) branch to instruction labeled L1
- **bne rs1, rs2, L1**
 - if (rs1 != rs2) branch to instruction labeled L1

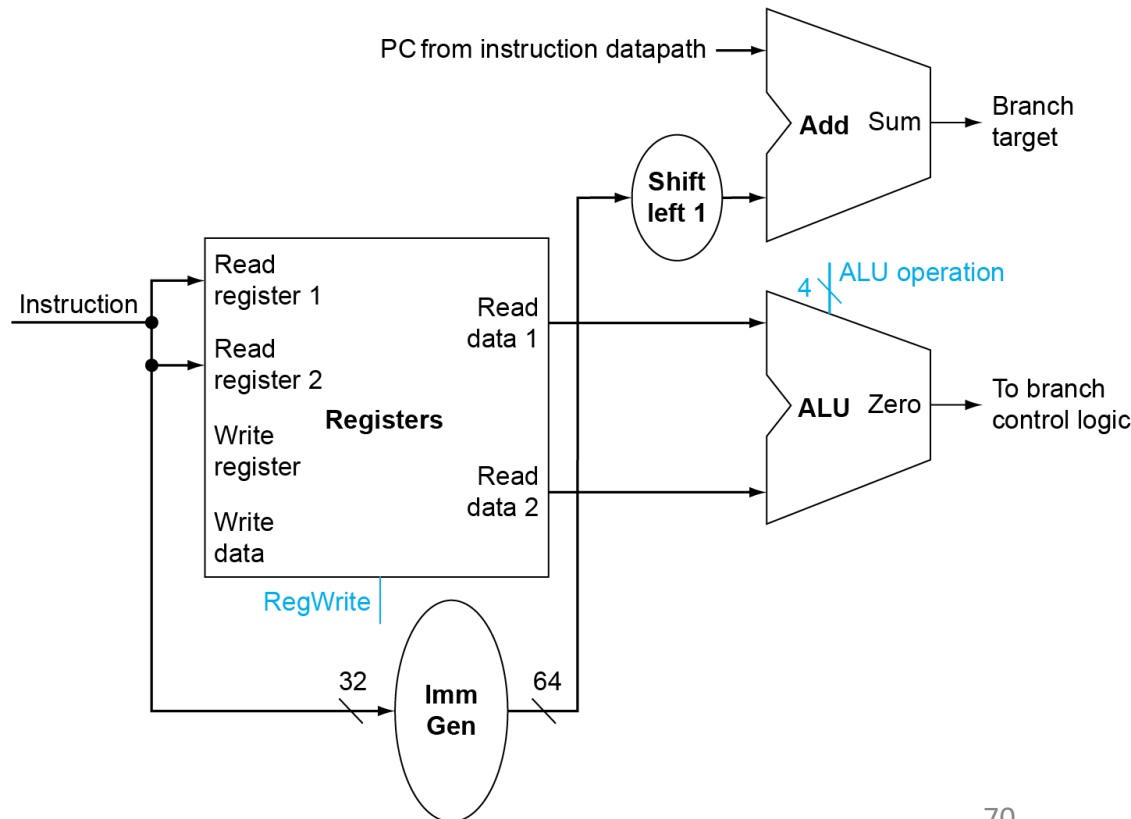
Branch If Equal (beq)

- **beq rs1, rs2, L1**
 - if (rs1 == rs2) branch to instruction labeled L1
 - beq rs1, rs2, L1 → beq rs1, rs2, offset
 - To calculate Branch Target Address, PC acts as the base address while a 12-bit offset field is available in the instruction.
 - The offset field is left-shifted 1-bit before being added to the PC to calculate the branch target address.
 - Why? The offset to the target instruction is supposed to represent the number of instructions (not the number of bytes) that we need to jump. If we restrict ourselves to 32-bit instructions (as in this course), then we need to shift by 2. However, shift by 1 is there to support the extension to RISC-V ISA that contains 16-bit compressed instructions.

Branch If Equal (beq)

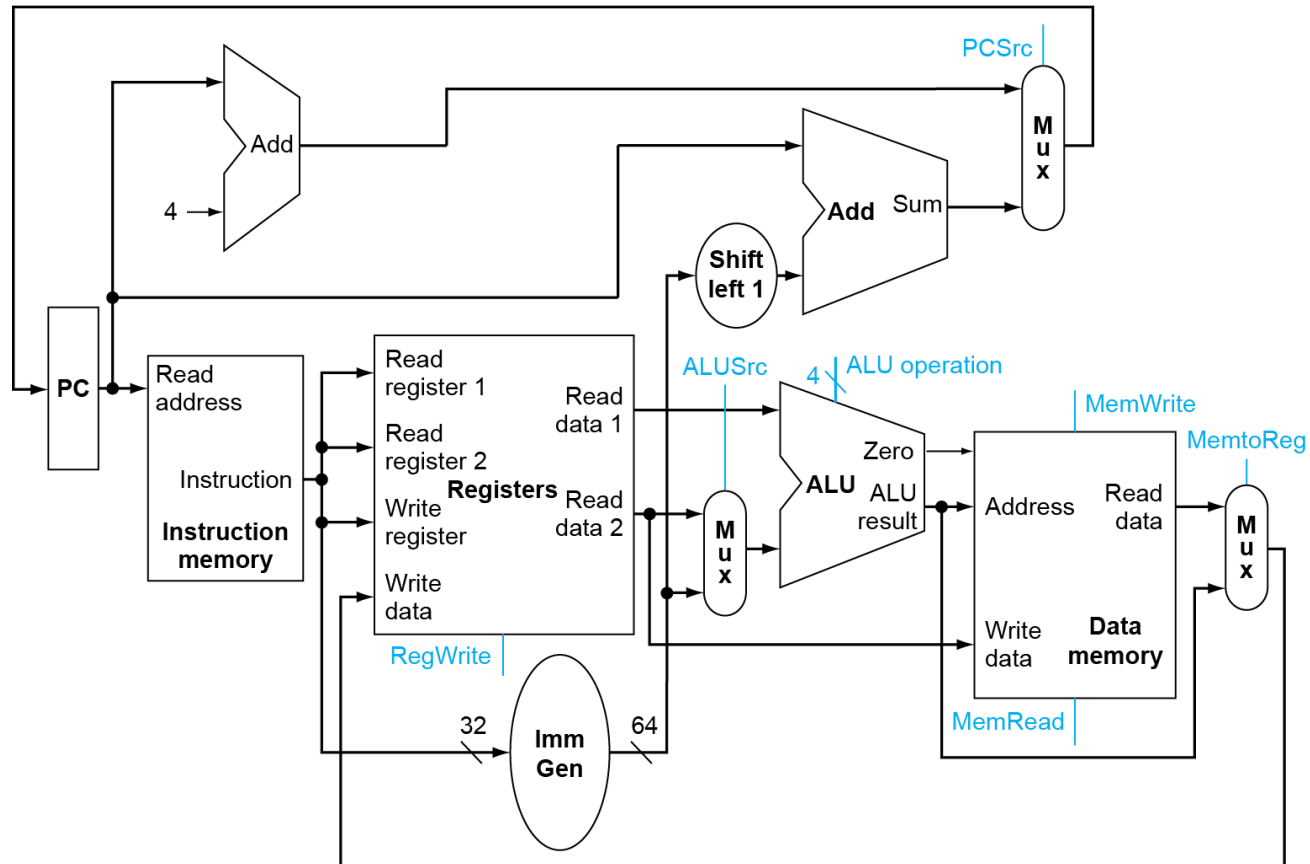
beq rs1, rs2, offset

- Read register operands
- Compare operands
 - Use ALU, subtract and check Zero output
- Calculate target address
- Sign-extend displacement
- Shift left 1 place (halfword displacement)
- Add to PC value



Building a Datapath: Composing the Elements

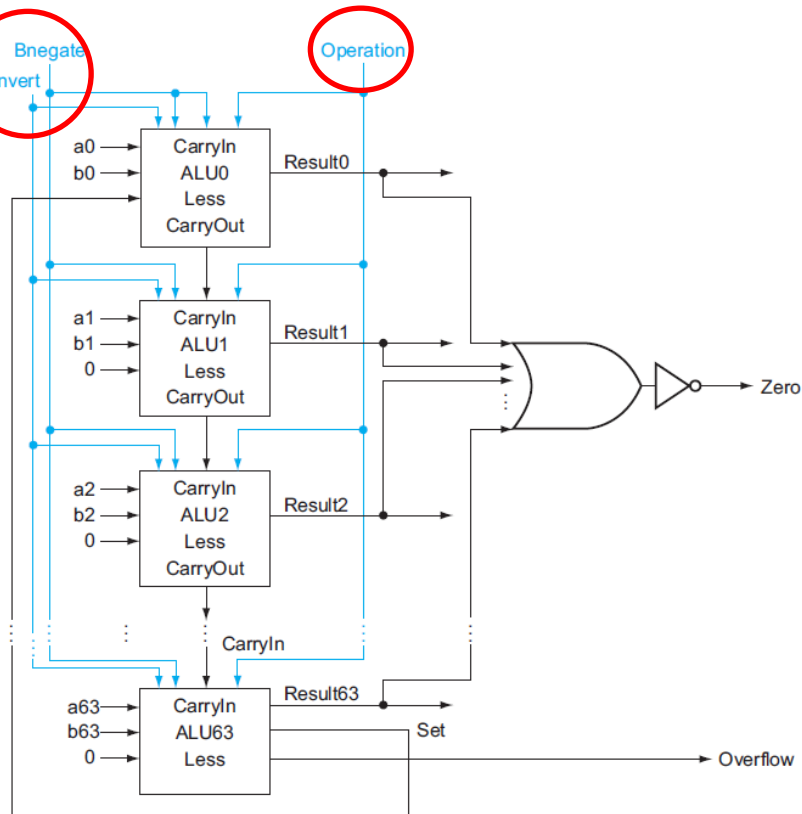
- Full Datapath



RISC-V Processor Implementation: Overview

- Datapath Design (Continued)
- **Control Lines and Control Unit**

Arithmetic Logic Unit (ALU)

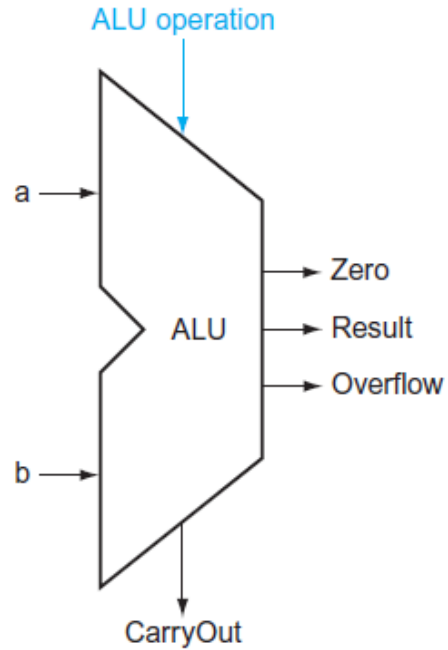


ALU Control

- We can think of the combination of the 1-bit Ainvert line, the 1-bit Bnegate line, and the 2-bit Operation lines as **4-bit control lines for the ALU**, telling it to perform add, subtract, AND, OR, NOR, or set less than.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

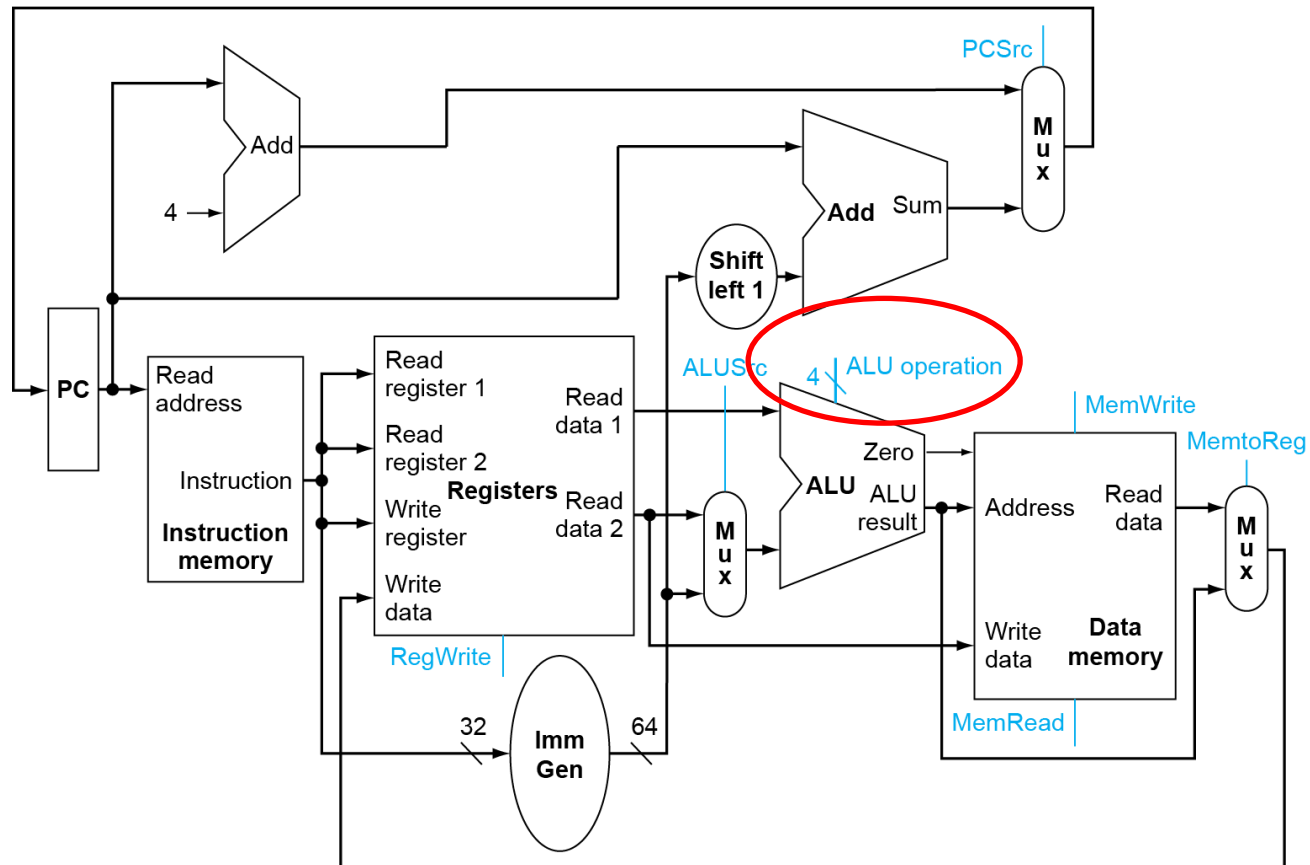
Arithmetic Logic Unit (ALU)



Symbol for a Complete ALU

Building a Datapath: Composing the Elements

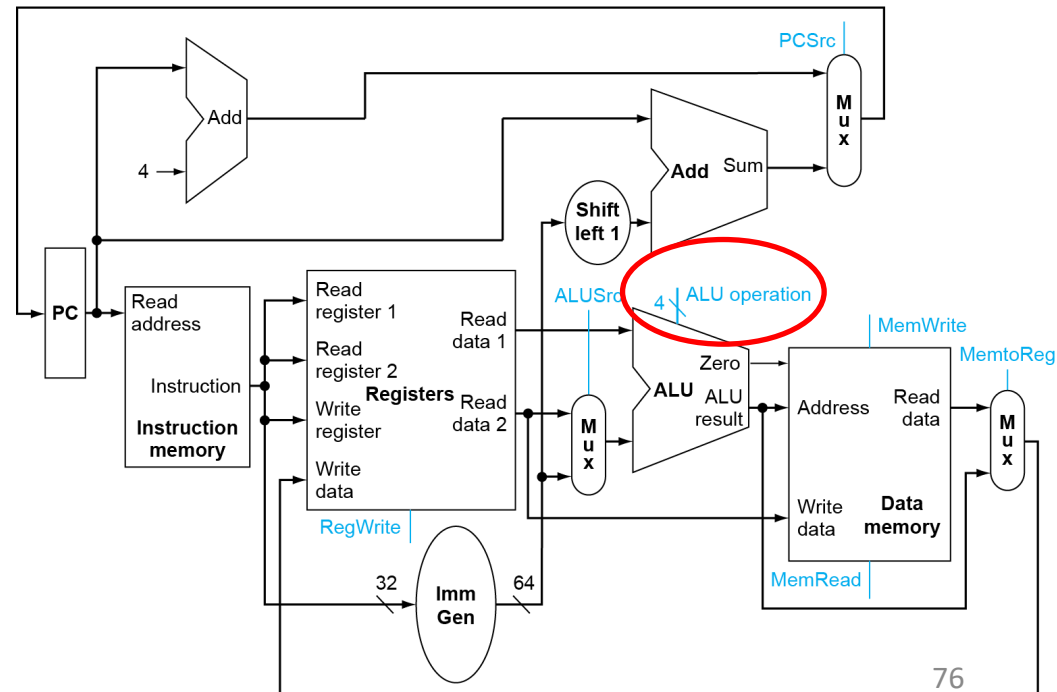
- Full Datapath



ALU Usage for Different Instructions

- Load-Store Instructions
 - ALU used for addition
- Branch If Equal (beq) Instruction
 - ALU used for subtraction
- R-type Instruction
 - ALU usage depends on opcode

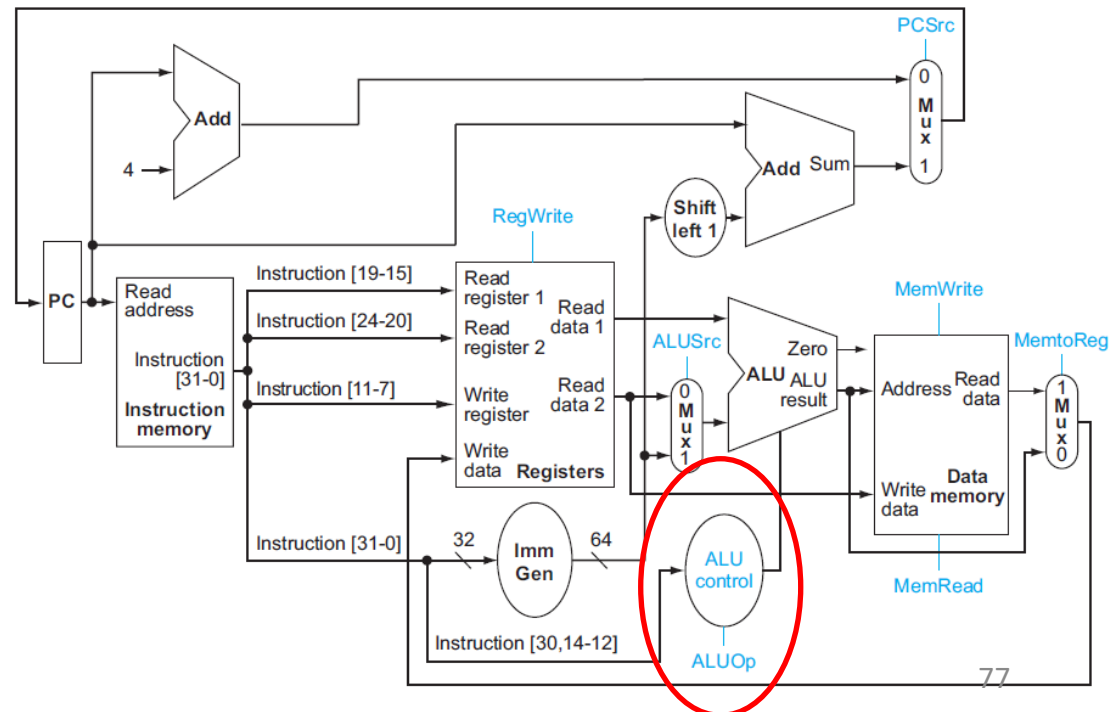
ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR



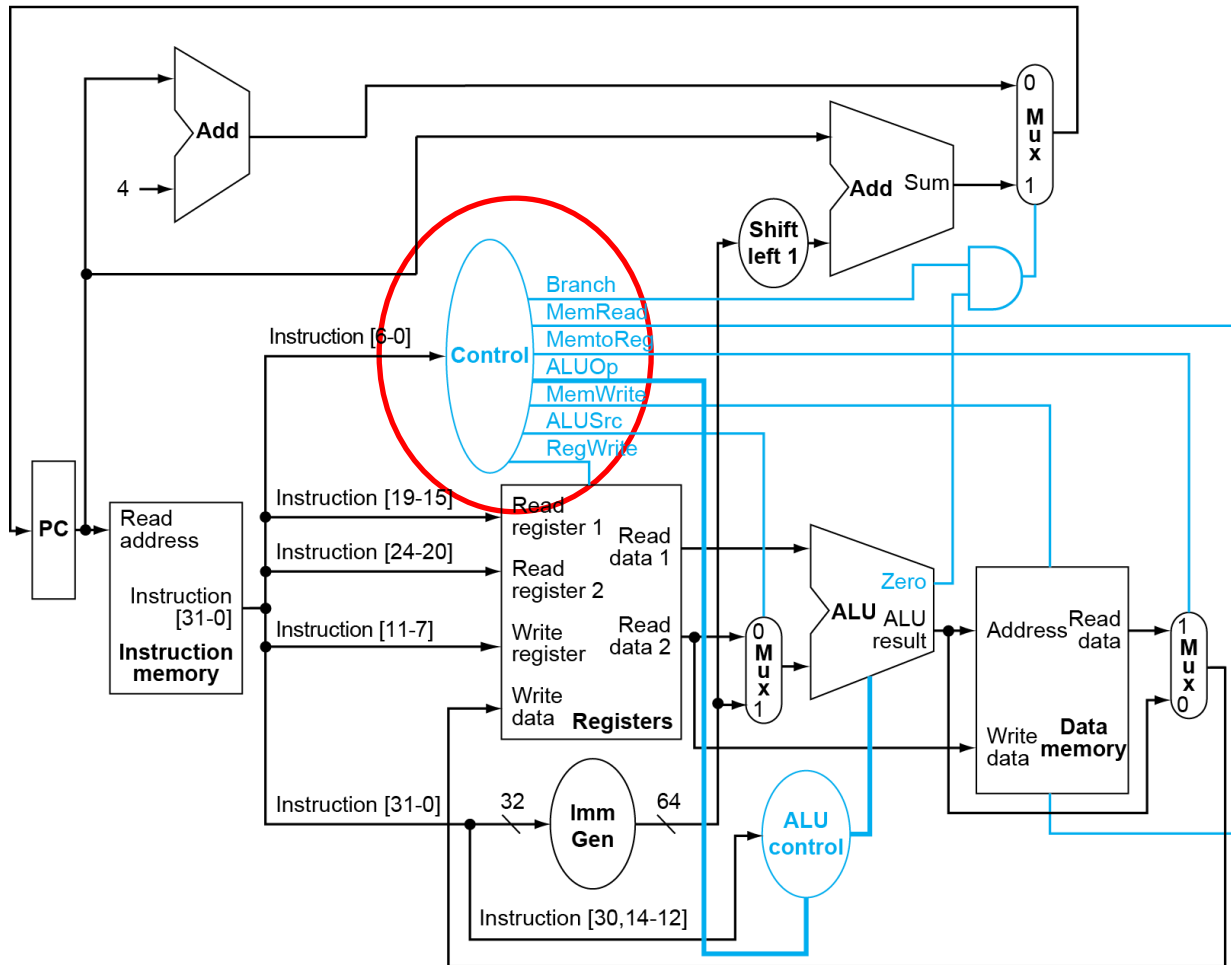
Generating the ALU Control Input

- We can generate the 4-bit ALU control input using a small control unit (ALUControl)
- Inputs to ALU Control
 - funct7 and funct3 fields of the instruction and a 2-bit control field, which we call ALUOp
- Output of ALU Control
 - 4-bit ALU control signal

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR



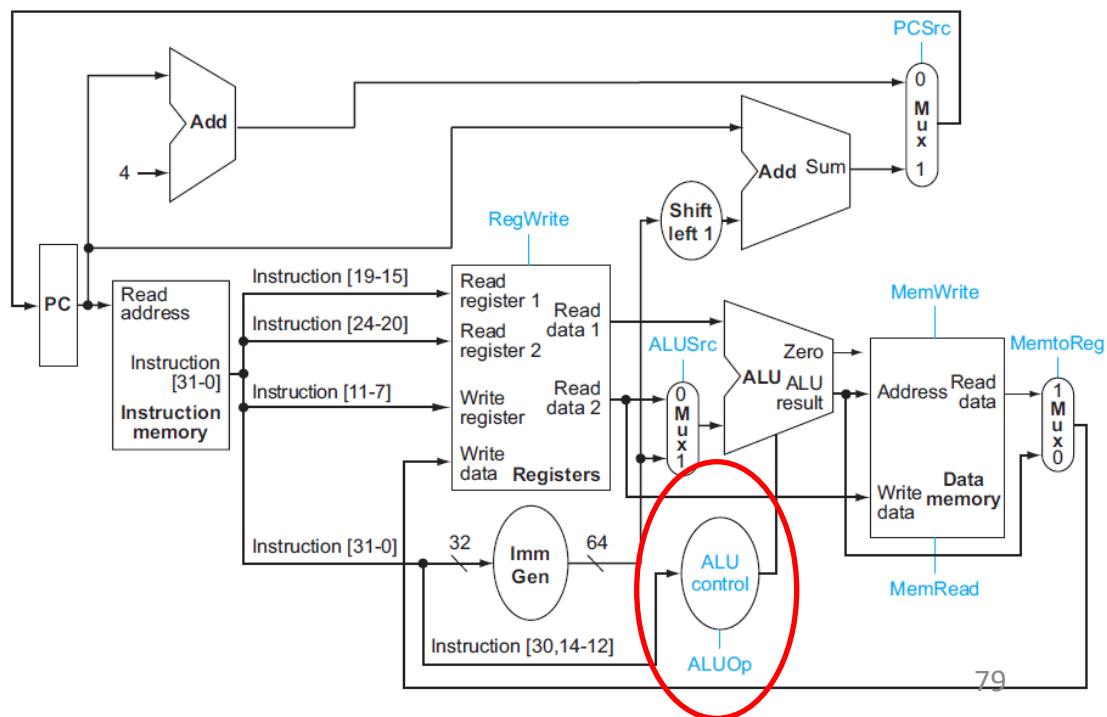
Datapath with Control



Generating the ALU Control Input

- We can generate the 4-bit ALU control input using a small control unit (ALUControl)
- Inputs to ALU Control
 - funct7 and funct3 fields of the instruction and a 2-bit control field, which we call ALUOp
- Output of ALU Control
 - 4-bit ALU control signal

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001



Generating the ALU Control Input

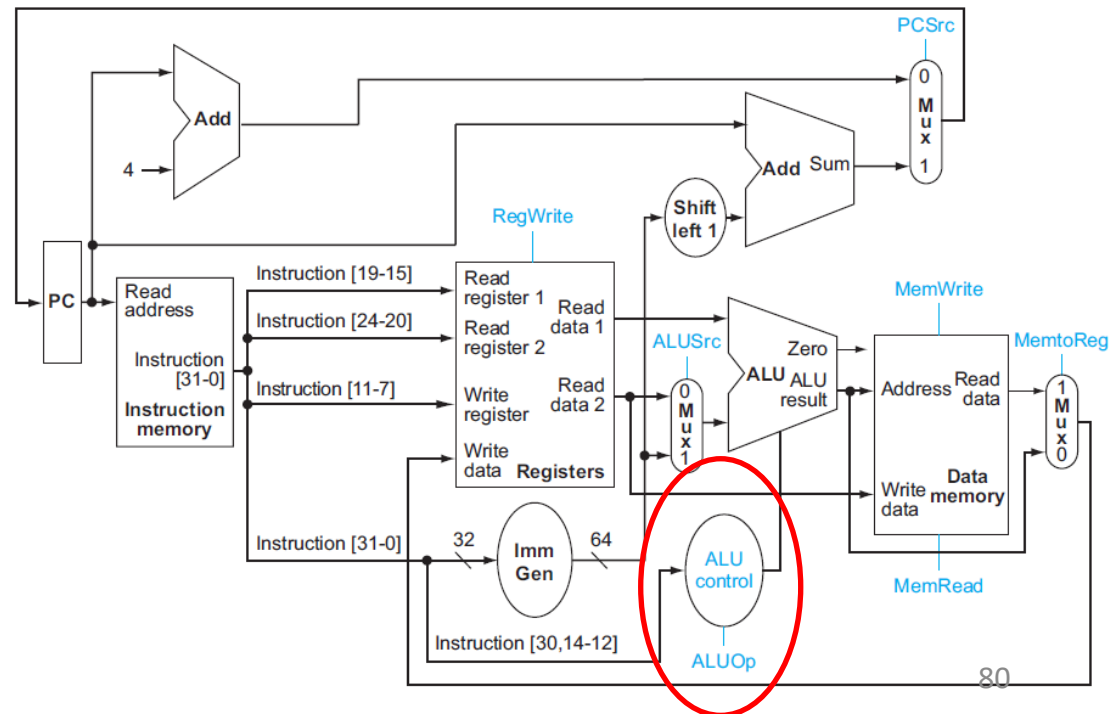
Truth Table

ALUOp		Funct7 field							Funct3 field			Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

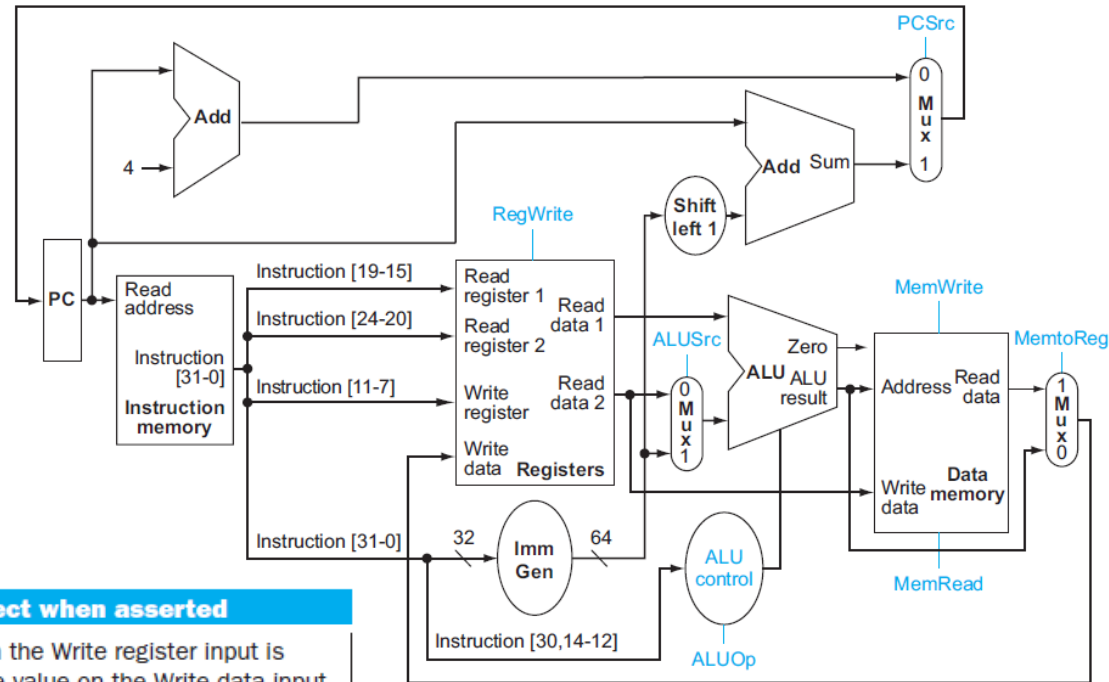
- We can generate the 4-bit ALU control input using a small control unit (ALUControl)

- Inputs to ALU Control
 - funct7 and funct3 fields of the instruction and a 2-bit control field, which we call ALUOp

- Output of ALU Control
 - 4-bit ALU control signal

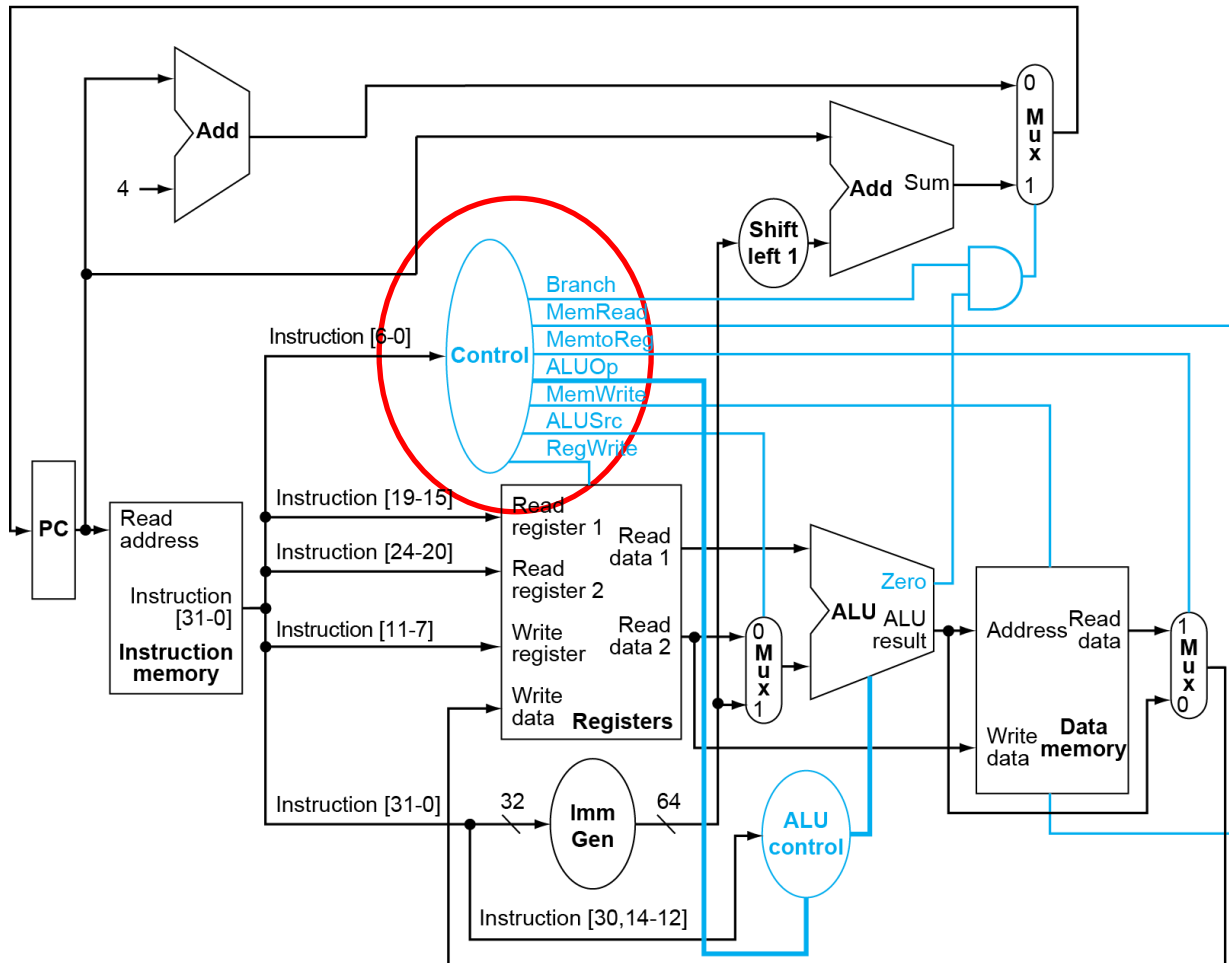


Control Signals Other than ALU

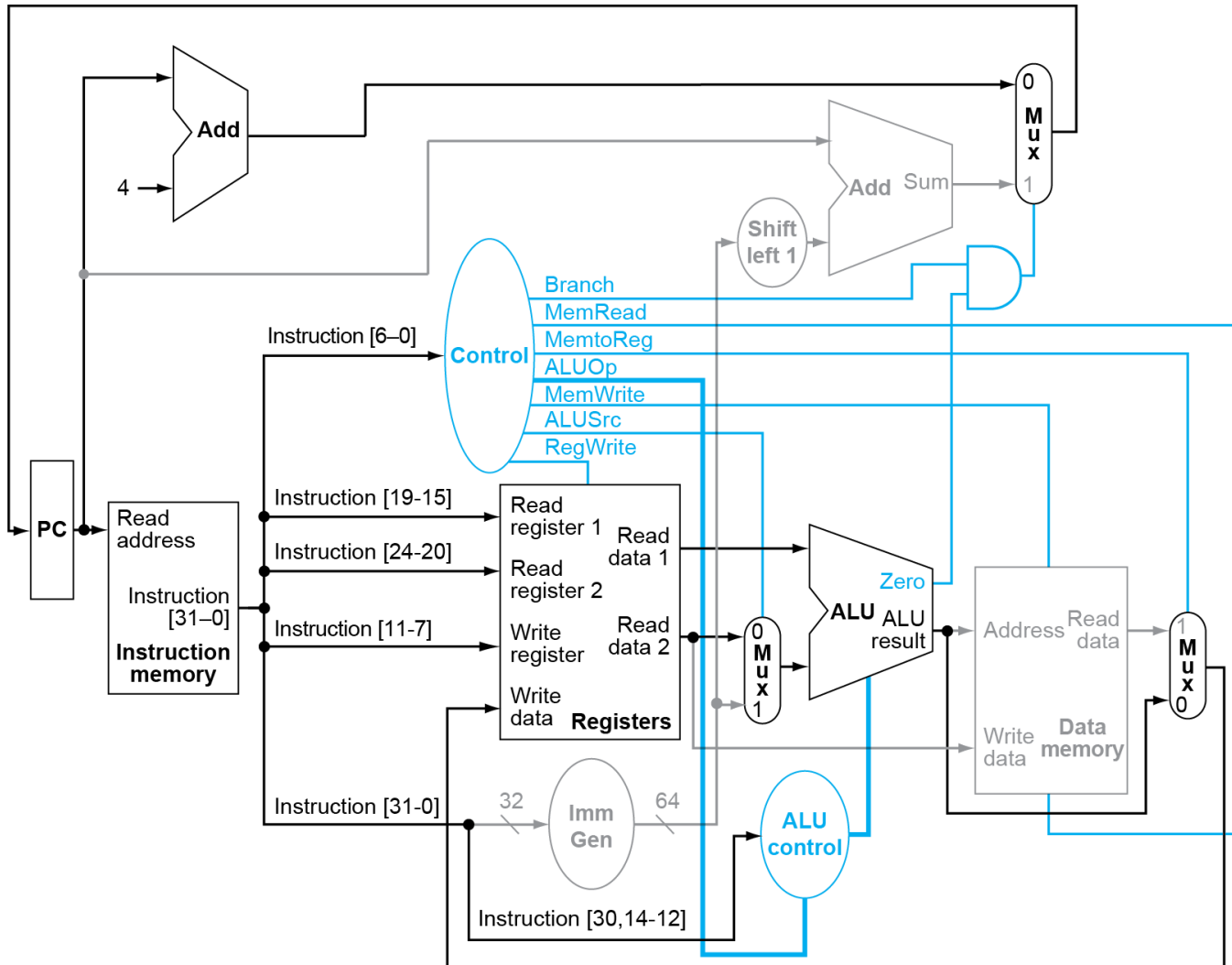


Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

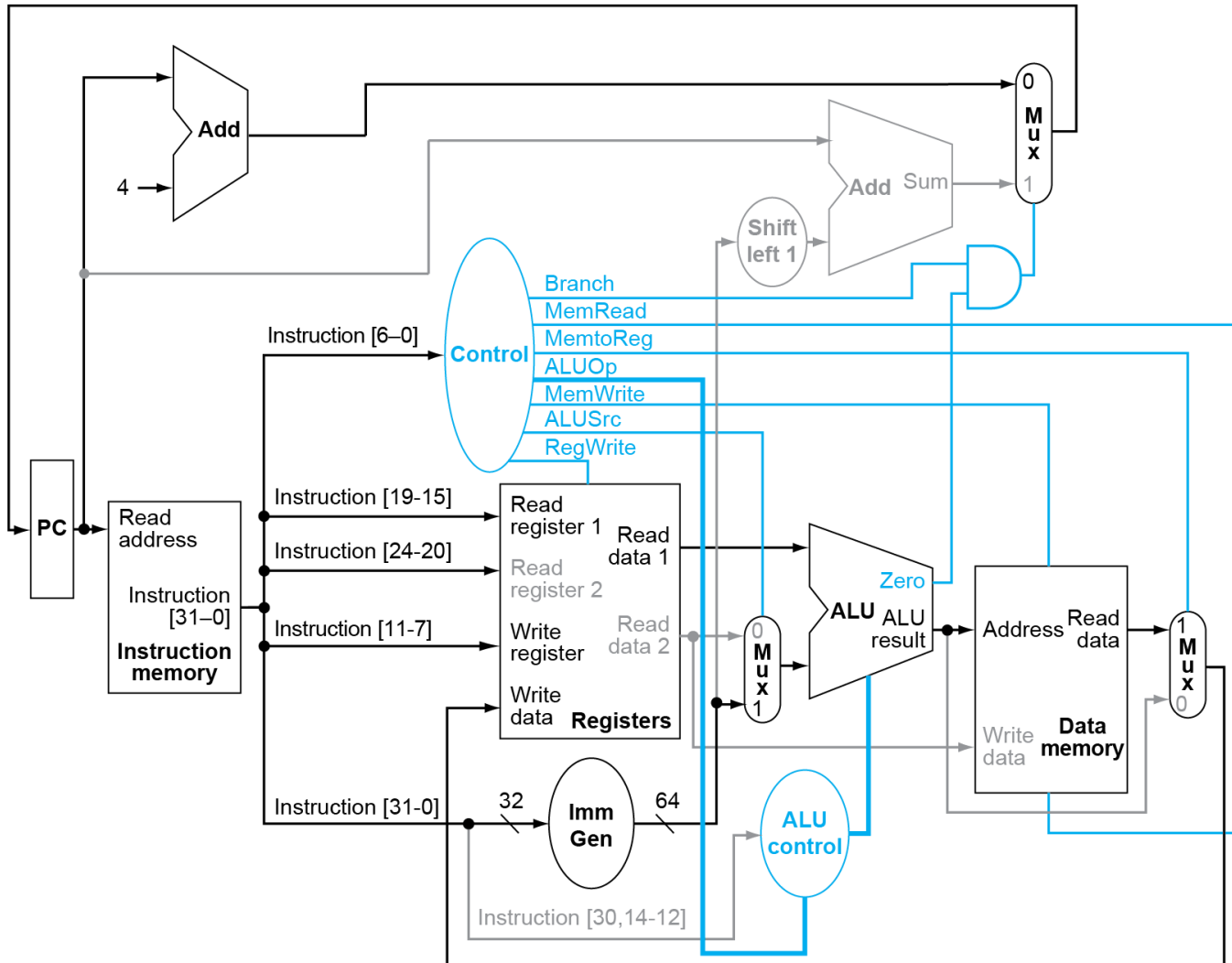
Main Control Unit



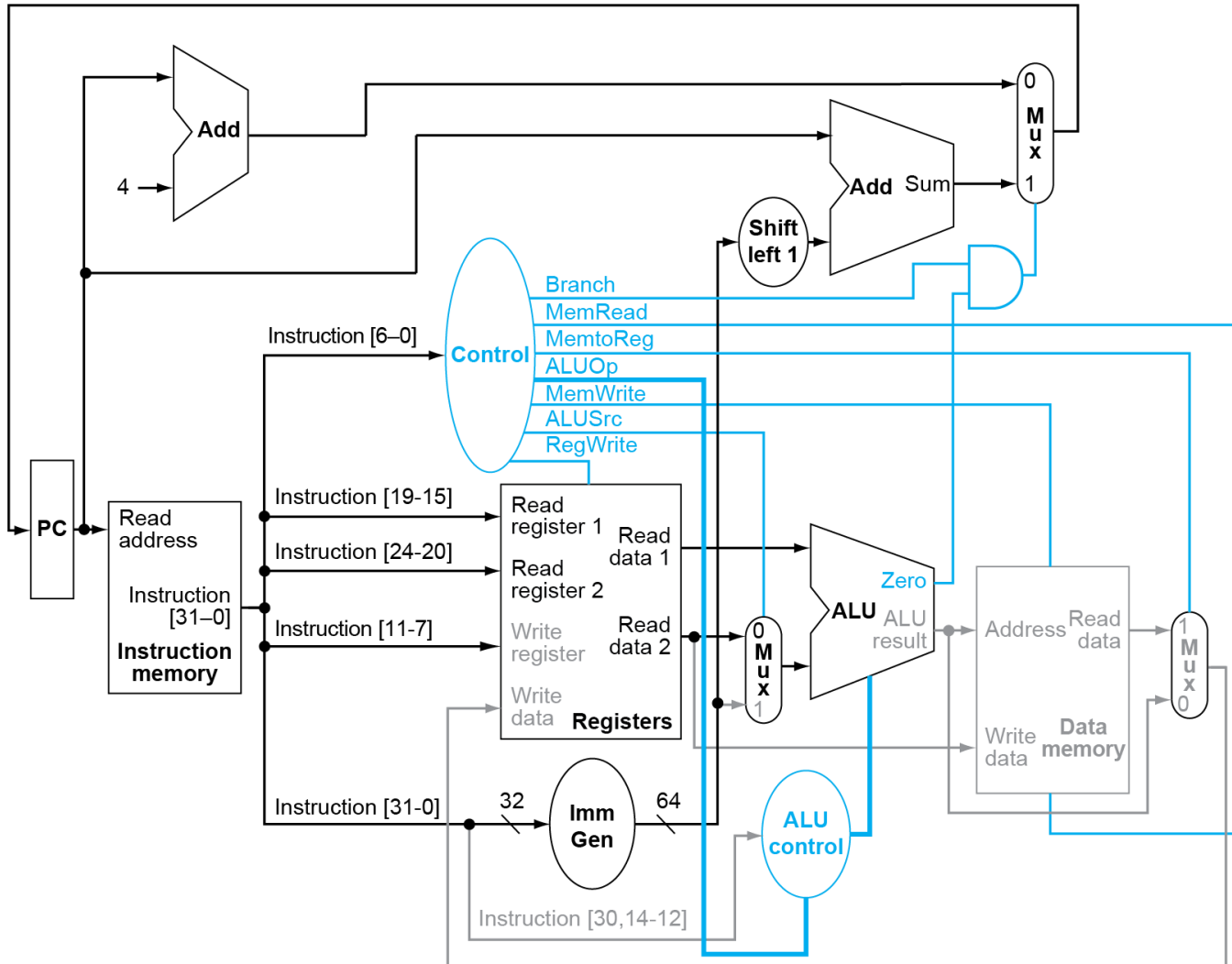
Datapath with Control: R-Type Instruction



Datapath with Control: Load Instruction



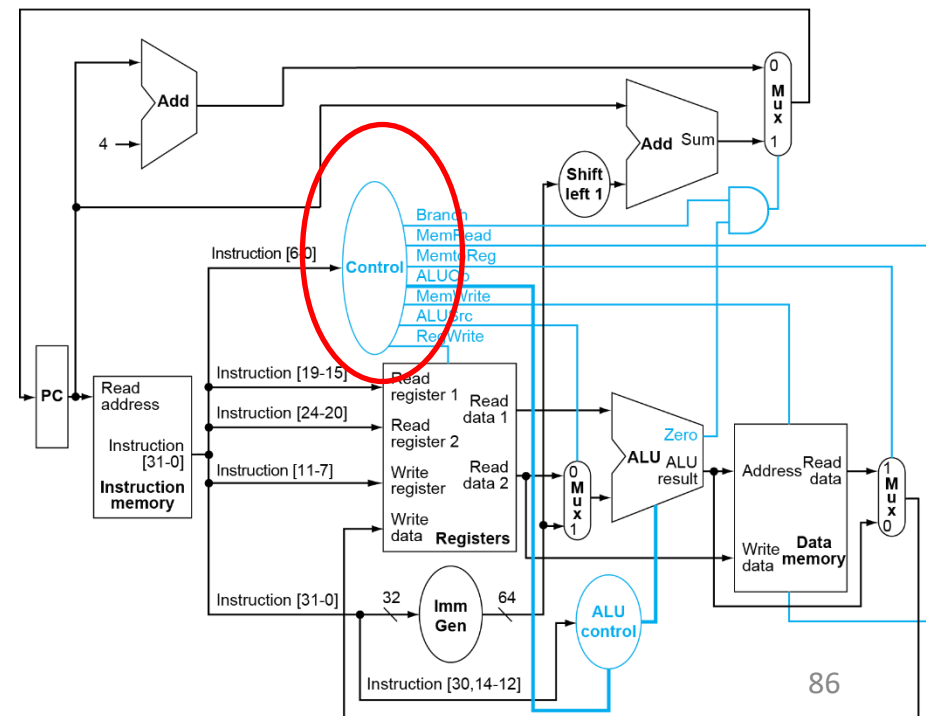
Datapath with Control: Beq Instruction



Implementation of Main Control Unit

- Control signals are derived from binary encoded instructions.
- The Main Control Unit can set all control signals, except *PCSrc*, based solely on the opcode and funct fields of the instruction.
- To generate *PCSrc* signal, *Branch* signal from the Main Control Unit is ANDed with the *Zero* signal from the ALU.

Name (Bit position)	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

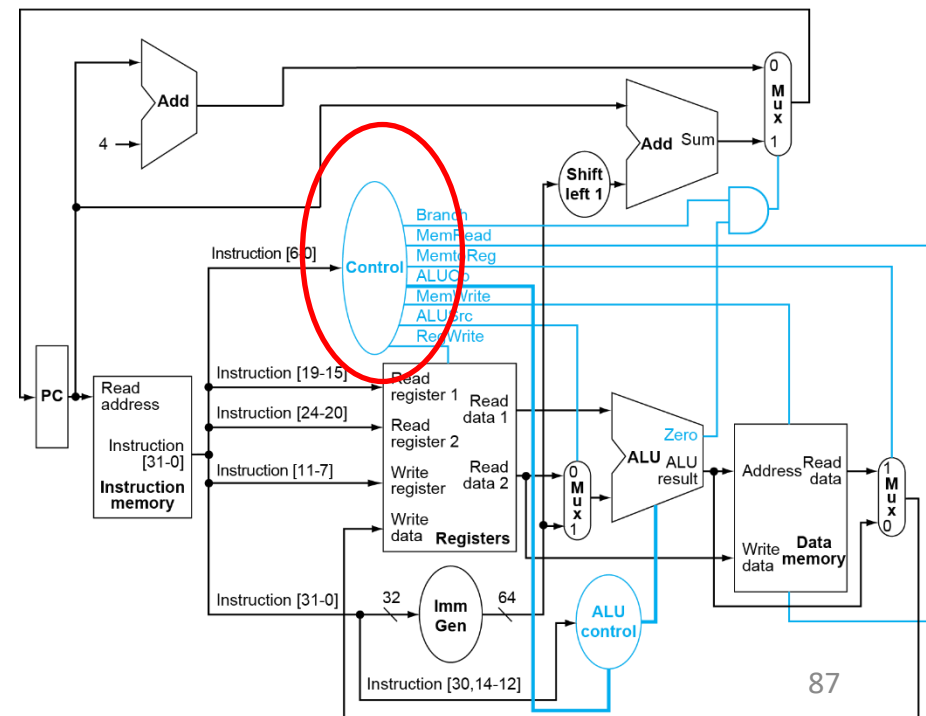


Implementation of Main Control Unit

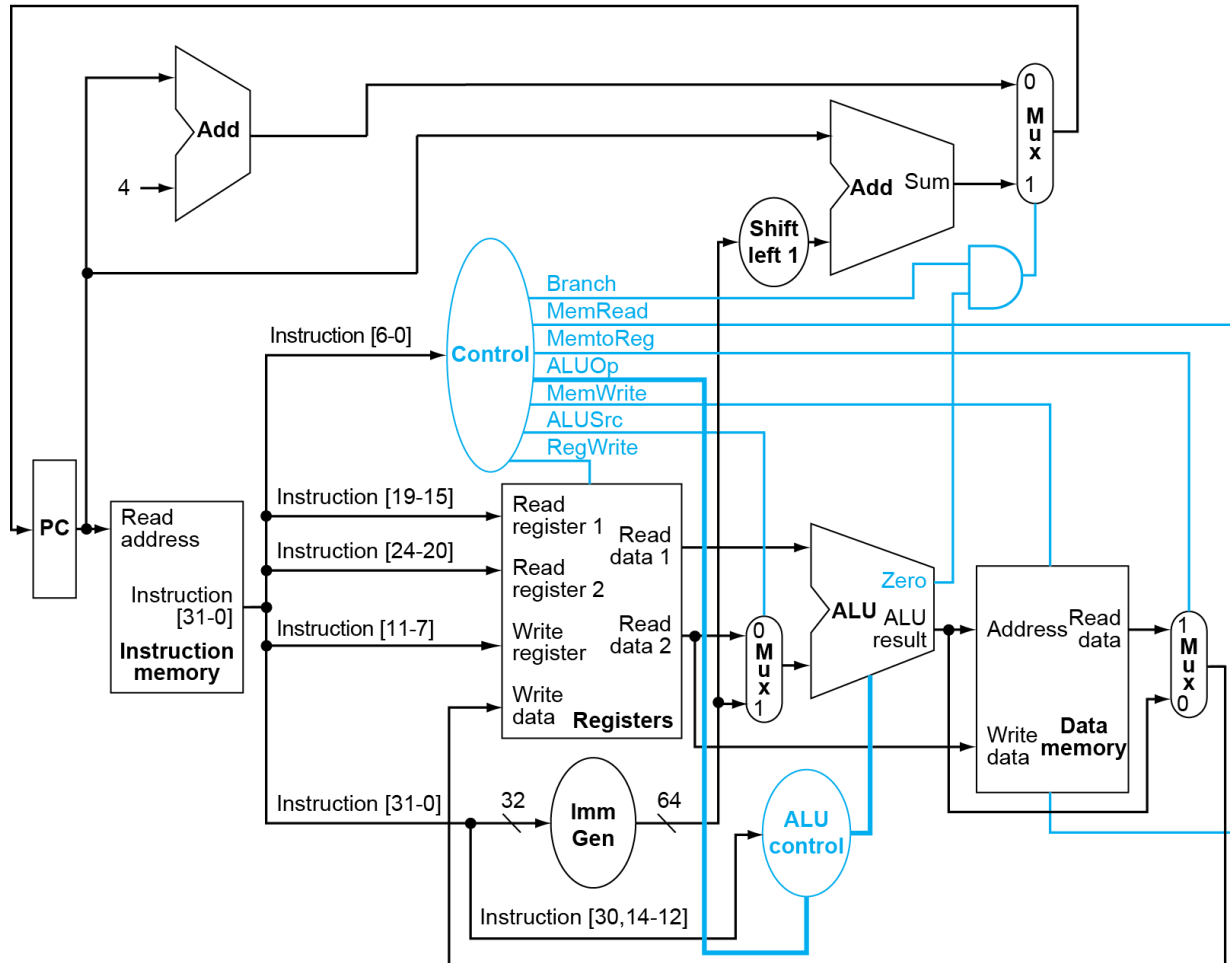
- Control signals are derived from binary encoded instructions.
- The Main Control Unit can set all control signals, except *PCSrc*, based solely on the opcode field of the instruction.
- To generate *PCSrc* signal, *Branch* signal from the Main Control Unit is ANDed with the *Zero* signal from the ALU.

Truth Table

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1



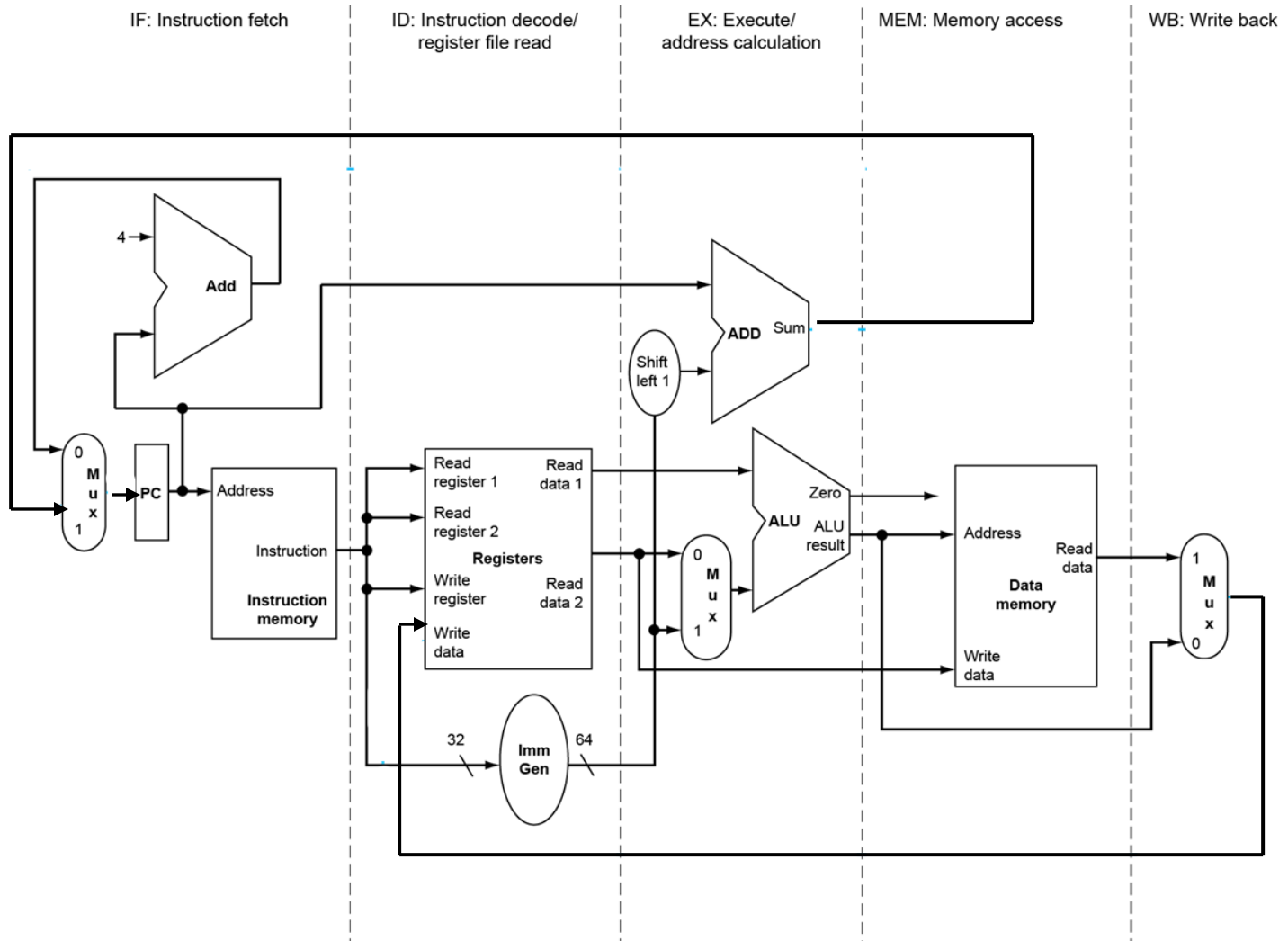
Datapath with Control



RISC-V Instruction Execution Steps

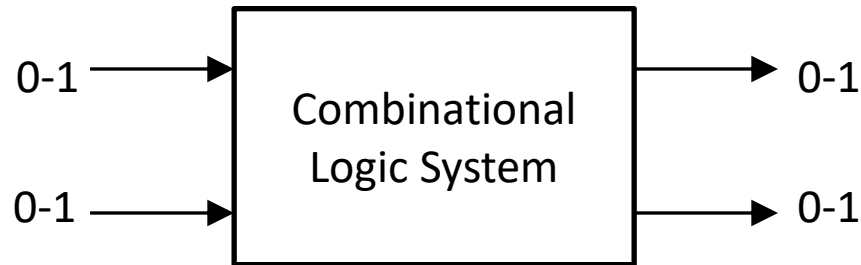
- RISC-V instructions classically take five steps:
 1. IF: Fetch instruction from memory.
 2. ID: Decode instruction and read registers
 3. EX: Execute the operation or calculate an address.
 4. MEM: Access an operand in data memory (if necessary).
 5. WB: Write the result back into a register (if necessary).

Steps in RISC-V Instruction Execution

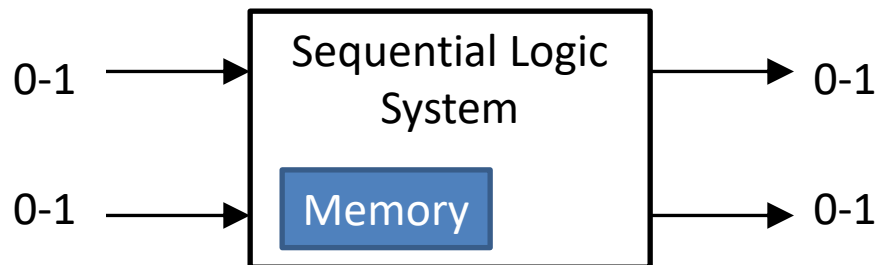


Review of Logic Design

- 2 Types of Logic Systems

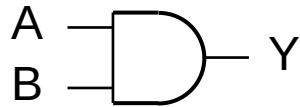


The output of a combinational logic system depends only on the current input.



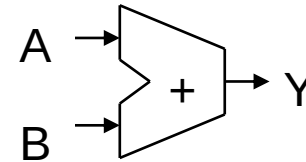
The output of a sequential logic system can depend both on the inputs and internal memory.

Combinational Logic Elements: Example



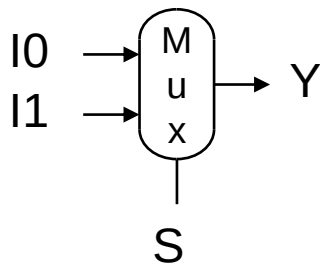
■ AND-gate

■ $Y = A \& B$



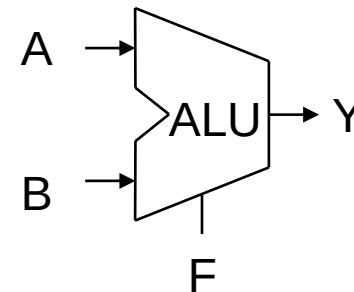
■ Adder

■ $Y = A + B$



■ Multiplexer

■ $Y = S ? I1 : I0$

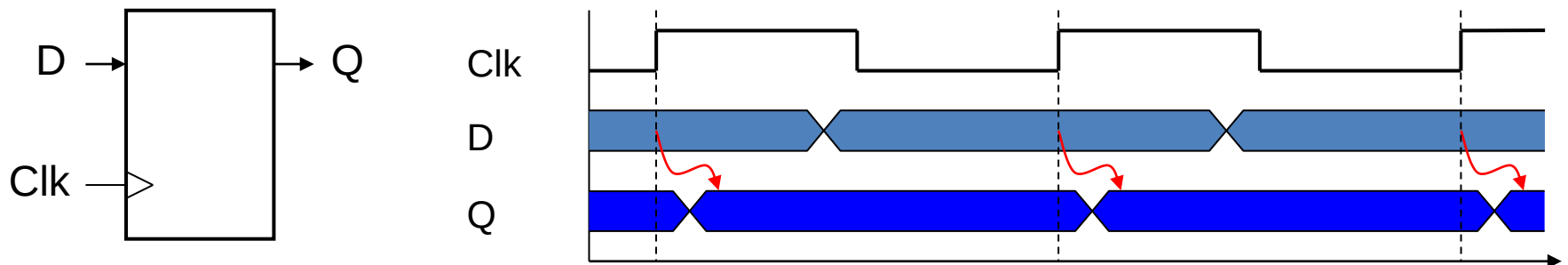


■ Arithmetic/Logic Unit

■ $Y = F(A, B)$

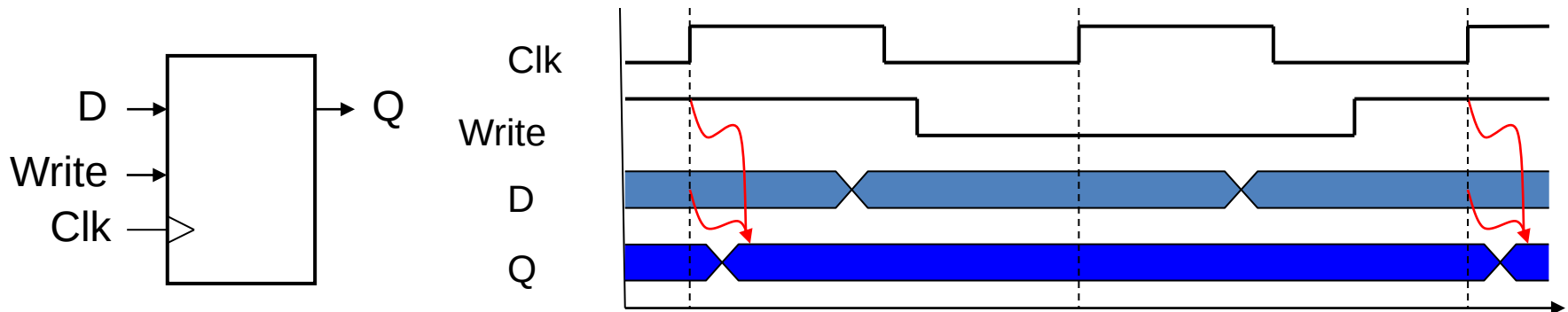
Sequential Logic (State) Elements: Example

- **Register**: stores data in a circuit
 - Uses a clock signal to determine when to update the stored value
 - Edge-triggered: update when Clk changes from 0 to 1



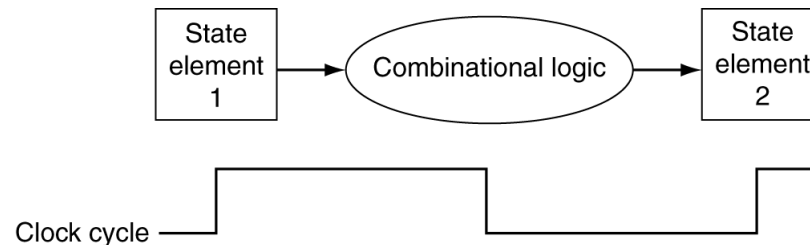
Sequential Logic (State) Elements: Example

- **Register with Write Control:**
 - Only updates on clock edge when write control input is 1
 - Used when stored value is required later

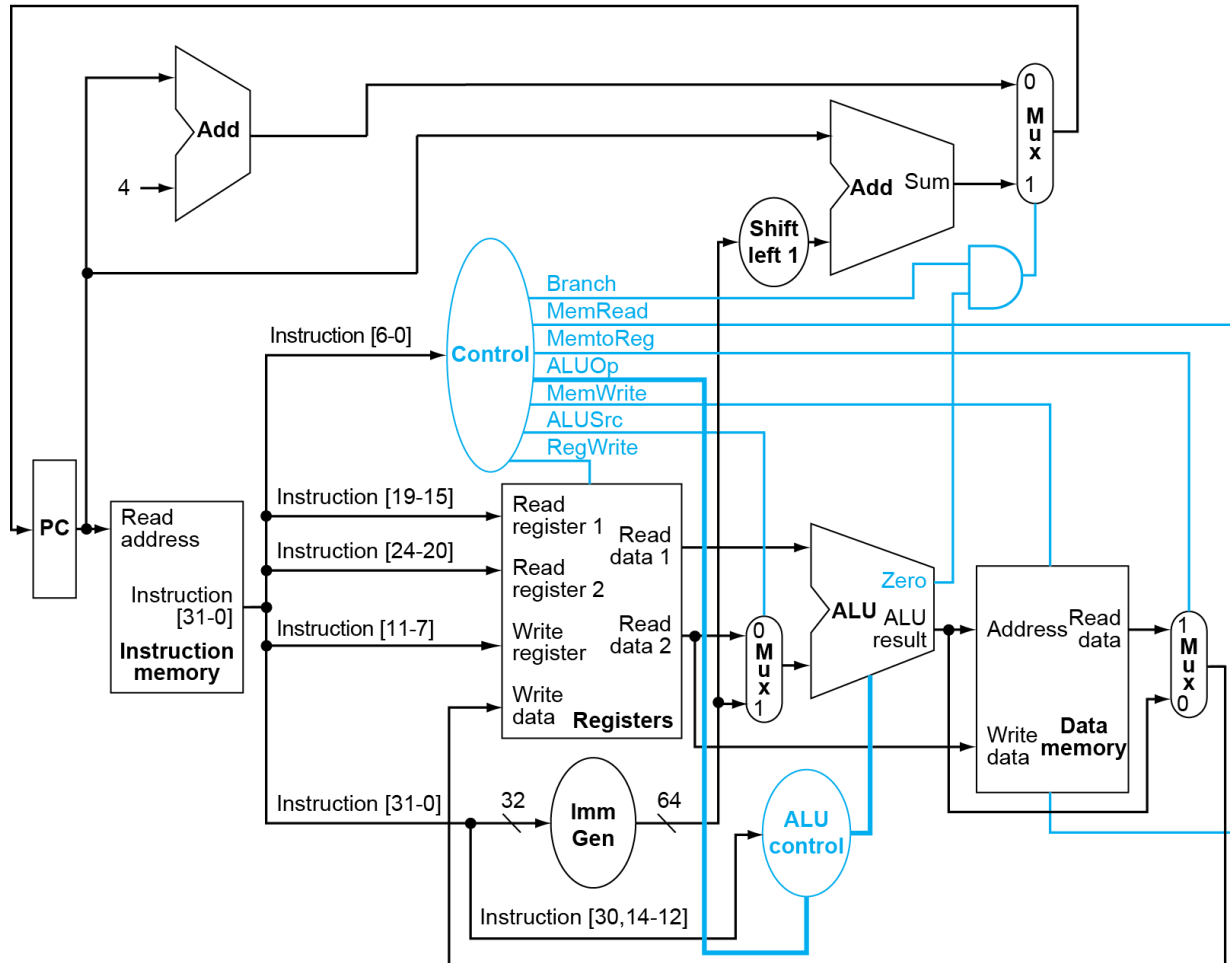


Combination of Combinational and Sequential Logic Elements

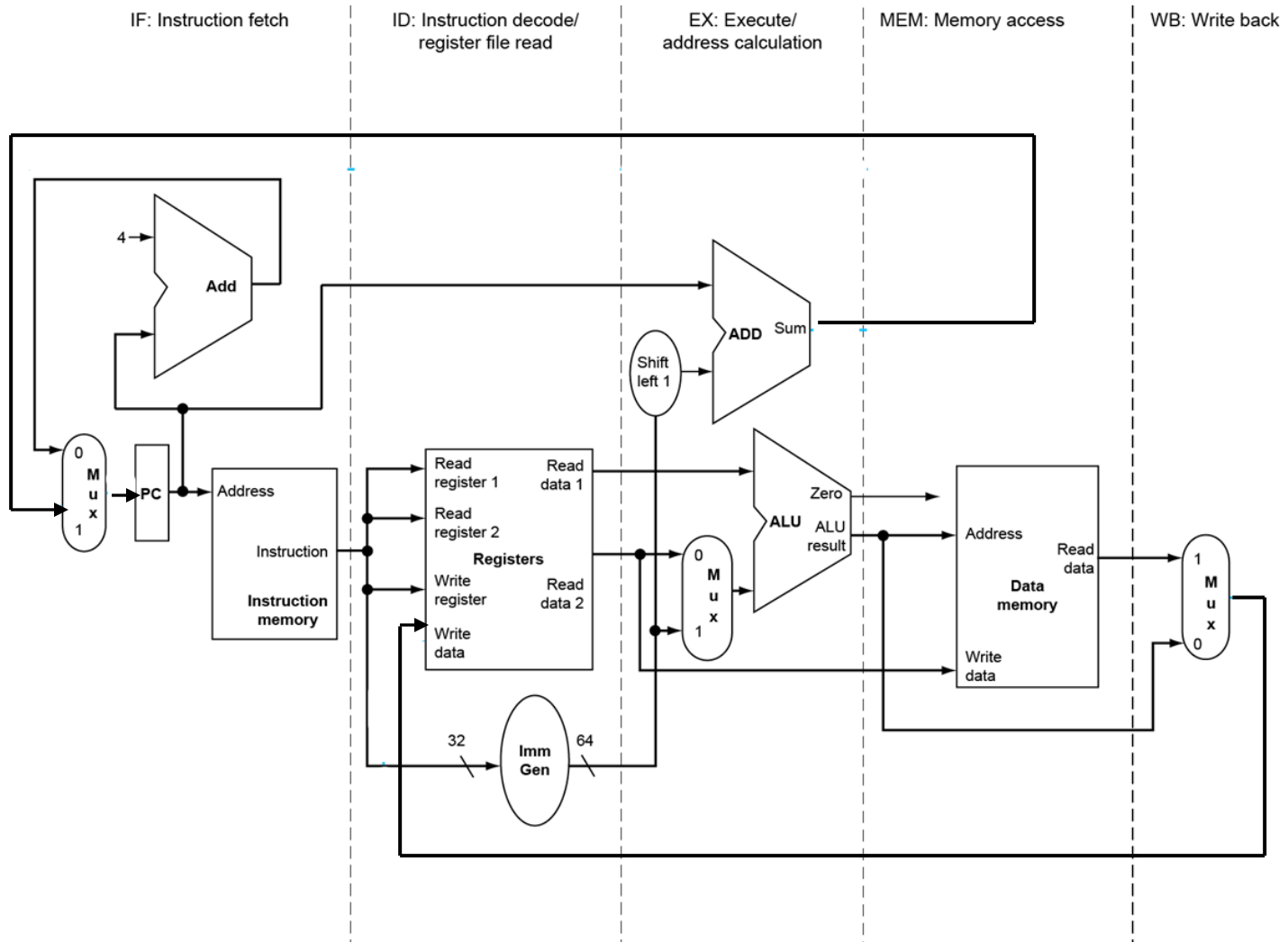
- Because only state elements can store a data value, any collection of combinational logic must have its inputs come from a set of state elements and its outputs written into a set of state elements
- When two state elements surround a block of combinational logic
 - All signals must propagate from state element 1, through the combinational logic, and to state element 2 in the time of one clock cycle.
 - The time necessary for the signals to reach state element 2 defines the length of the clock cycle



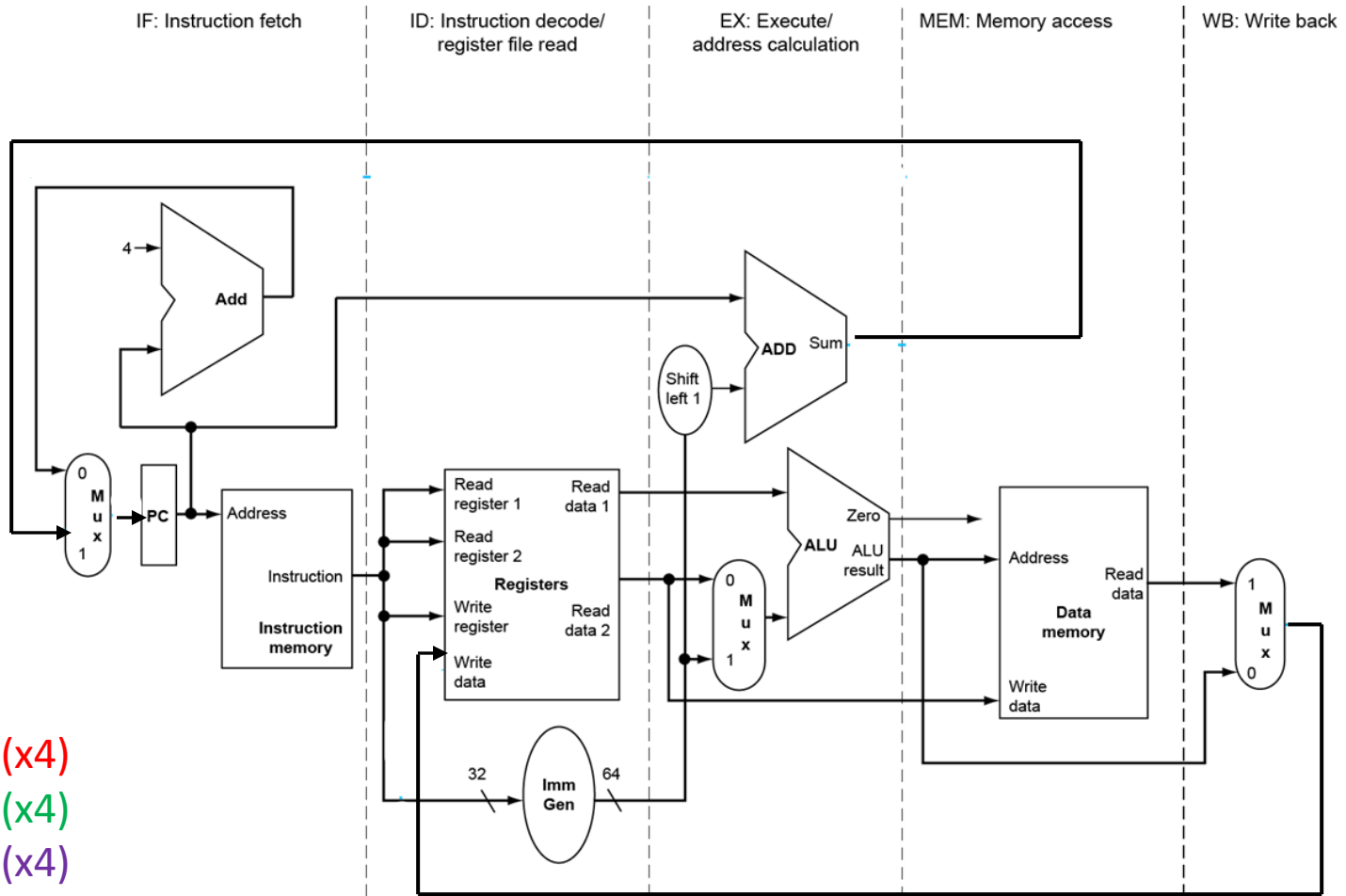
Datapath with Control



Steps in RISC-V Instruction Execution

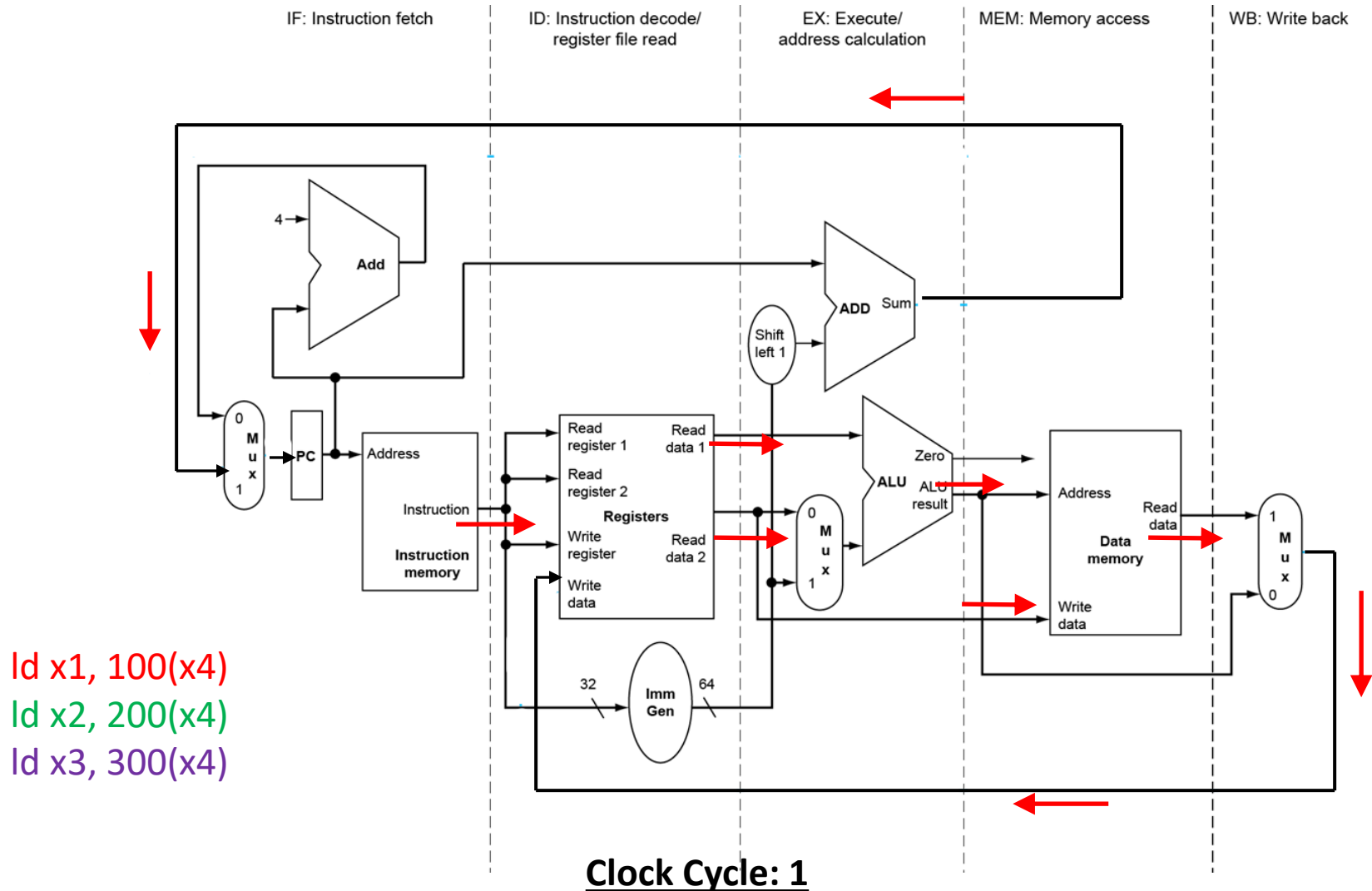


Steps in RISC-V Instruction Execution

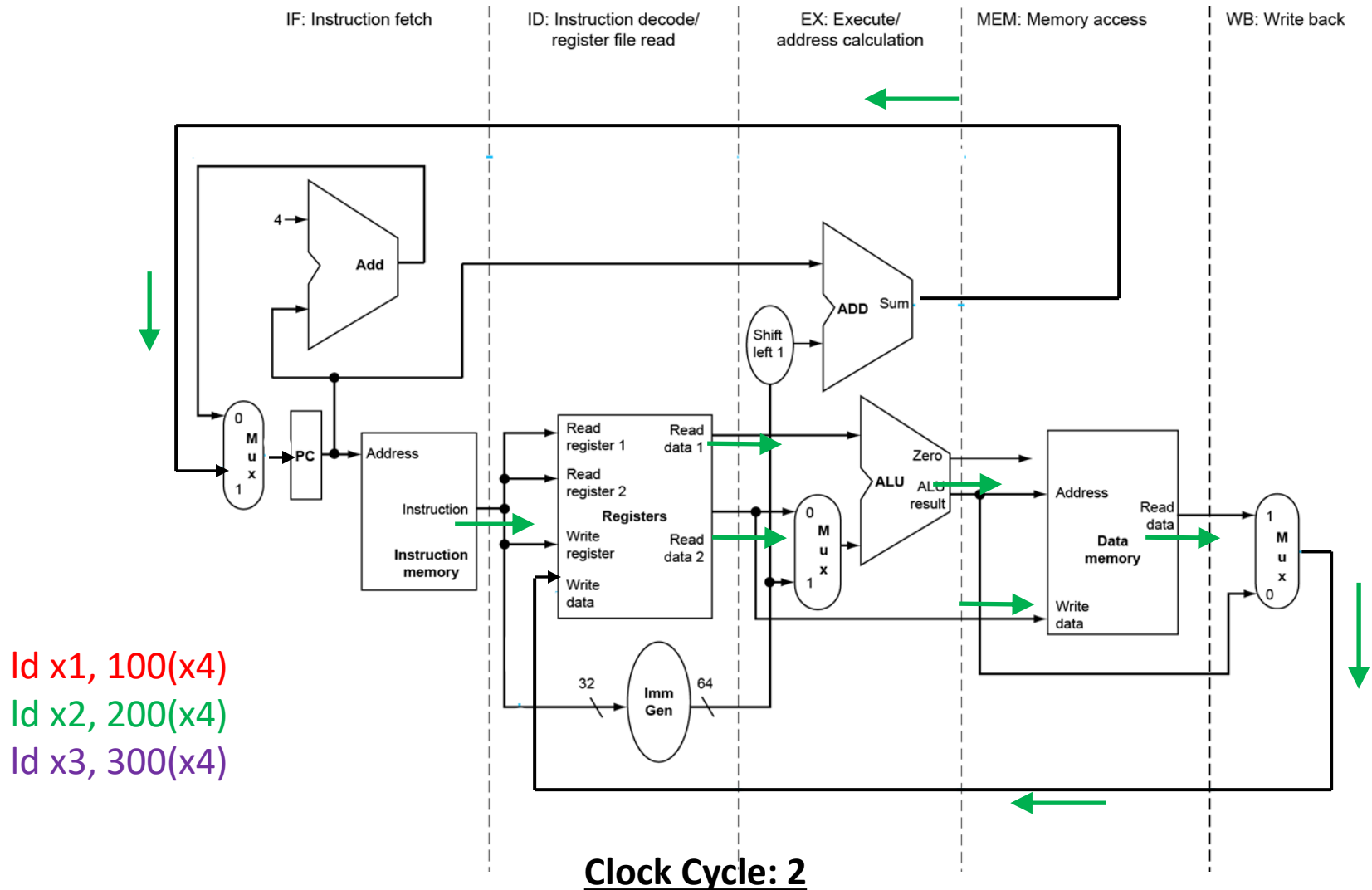


ld x1, 100(x4)
ld x2, 200(x4)
ld x3, 300(x4)

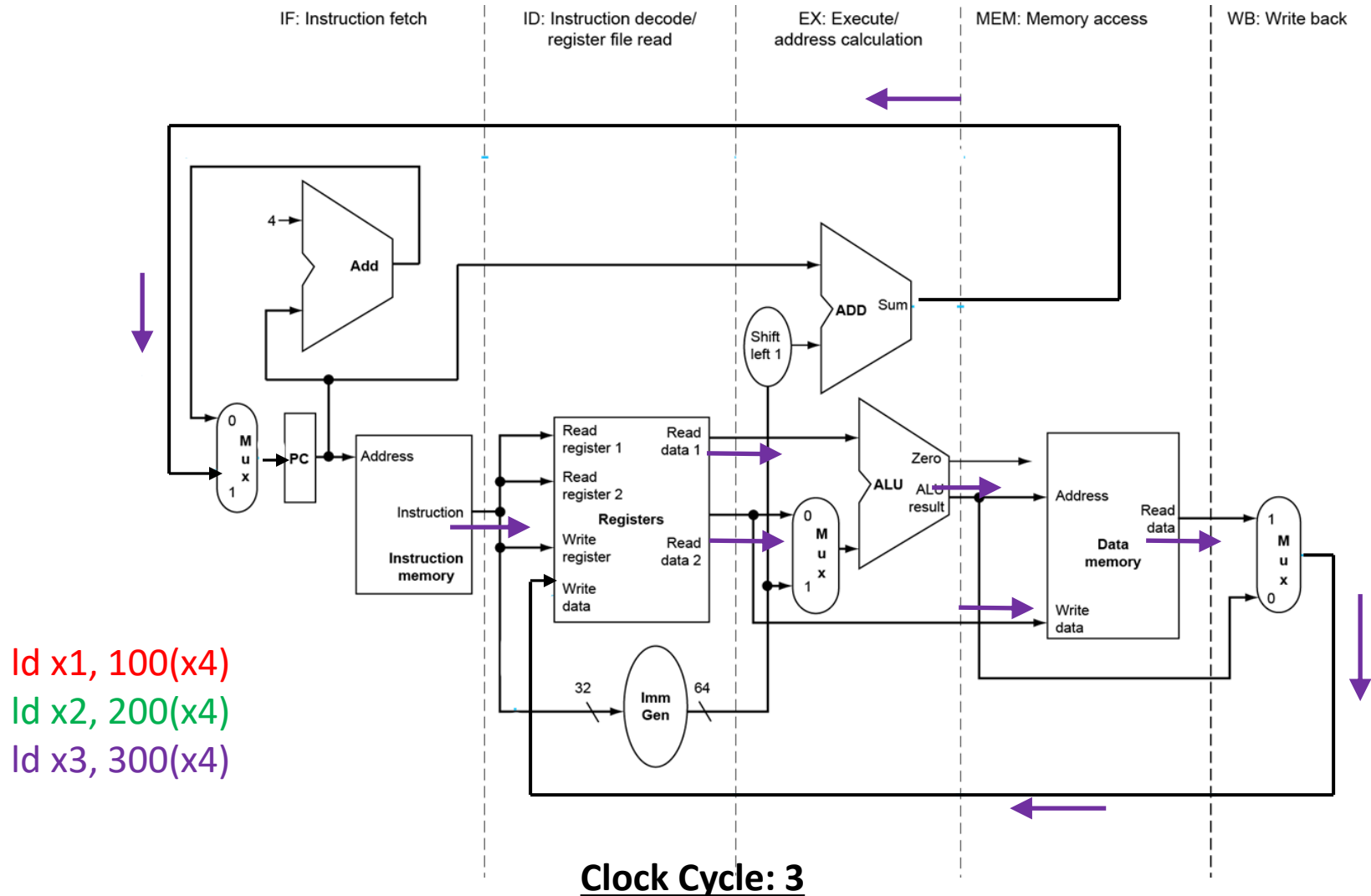
Steps in RISC-V Instruction Execution



Steps in RISC-V Instruction Execution



Steps in RISC-V Instruction Execution



Processor Implementation

- Classical performance equation:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

where

Instruction Count = Number of machine instruction in a program

CPI = Clock cycles per instruction

Clock cycle time = Clock Period = Time period of the clock signal

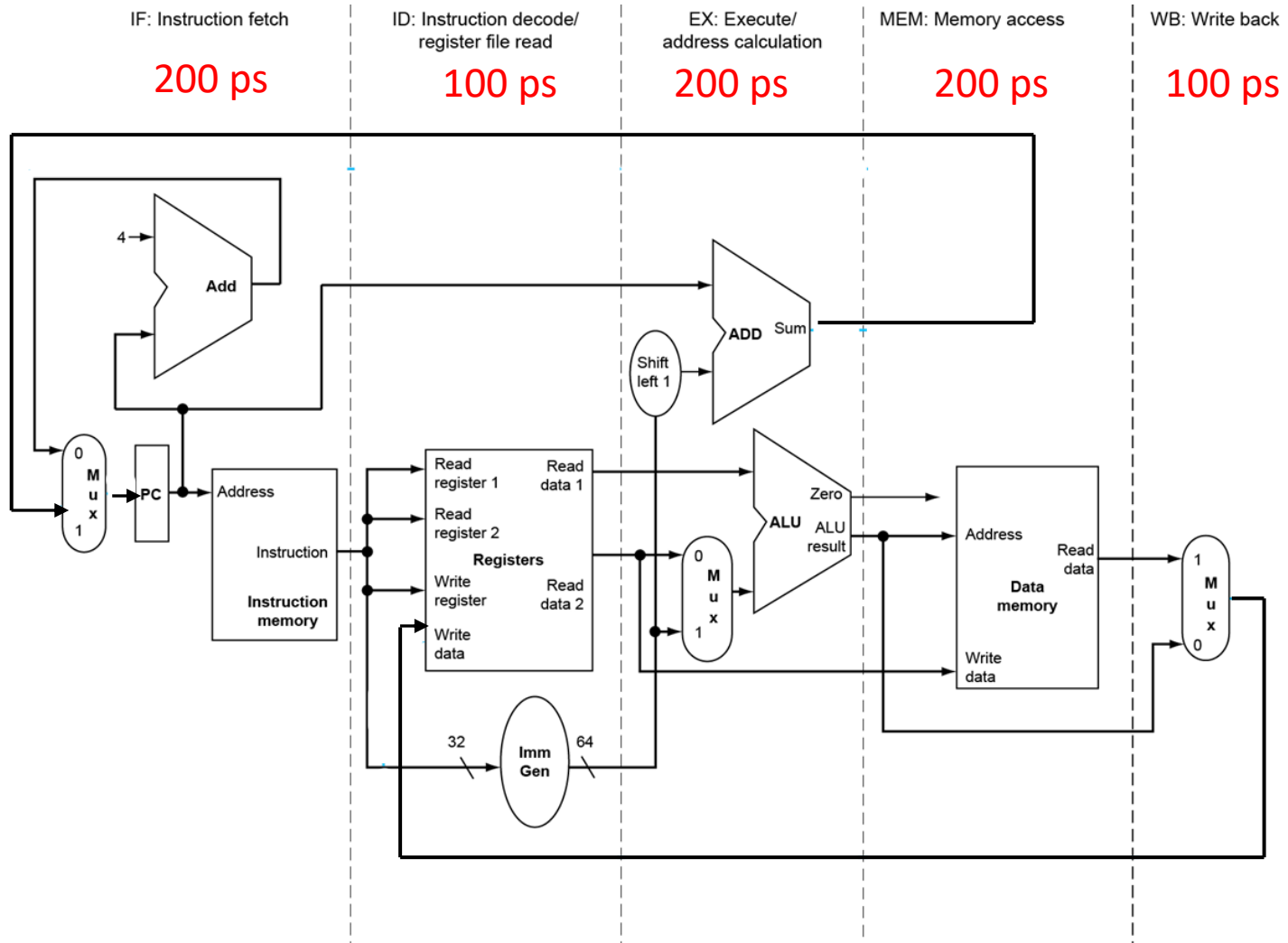
- Instruction Count
 - Determined by Compiler and Instruction Set Architecture
- CPI and Clock cycle time
 - Determined by Processor Implementation

Performance Issues of Single Cycle Processor Design

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

- CPI is 1.
- However, clock cycle time is determined by the longest possible path/delay in the data path.
 - Critical Path: Load Instruction
 - Load instruction uses five functional units in series: Instruction memory → register file → ALU → data memory → register file

Steps in RISC-V Instruction Execution



RISC-V Performance: Single Cycle

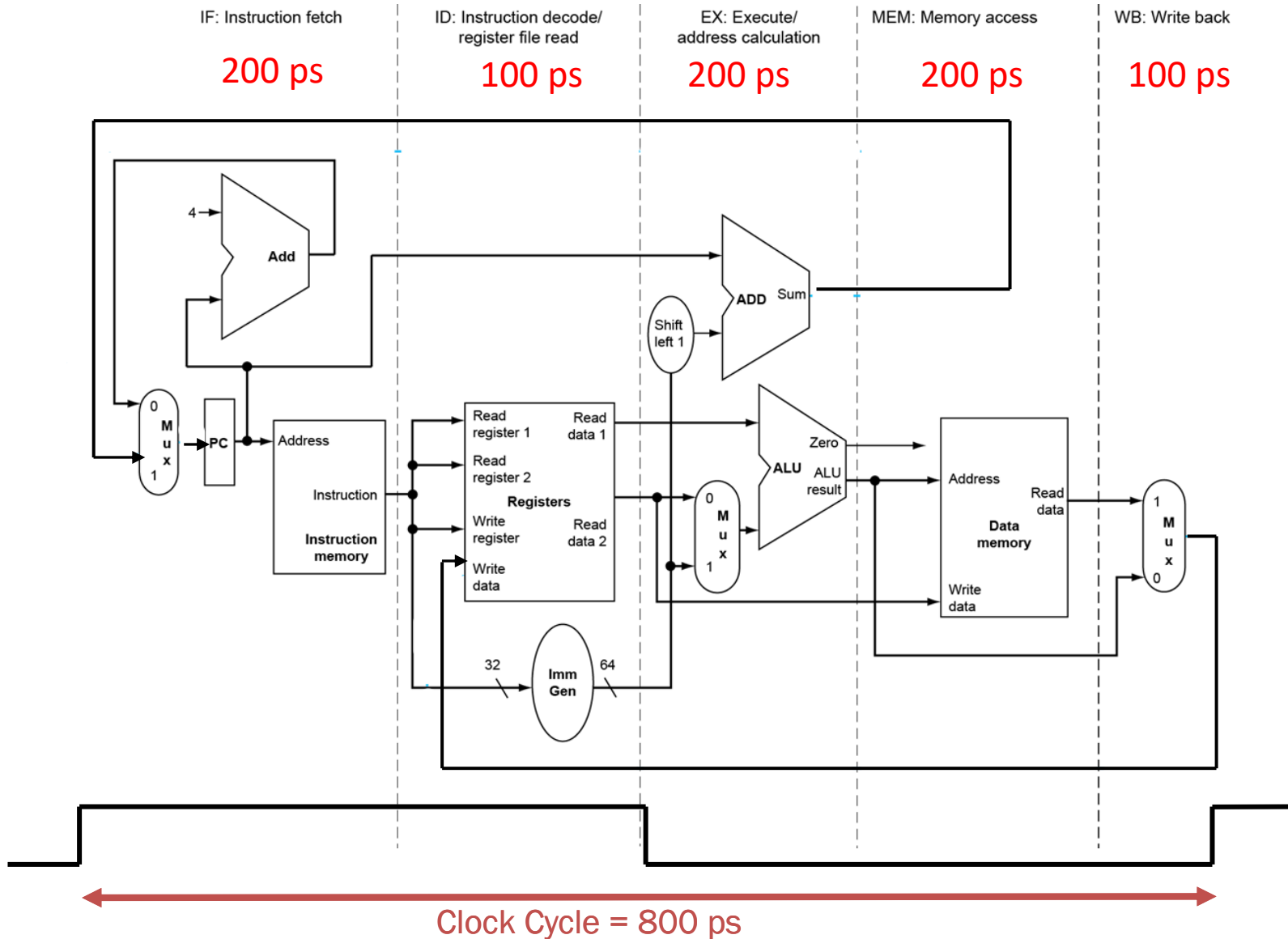
- Assume that the operation times for the major functional units of data path are :
 - 100ps for register read or write
 - 200 ps for memory access for instructions or data,
 - 200 ps for ALU operation

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load doubleword (ld)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store doubleword (sd)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

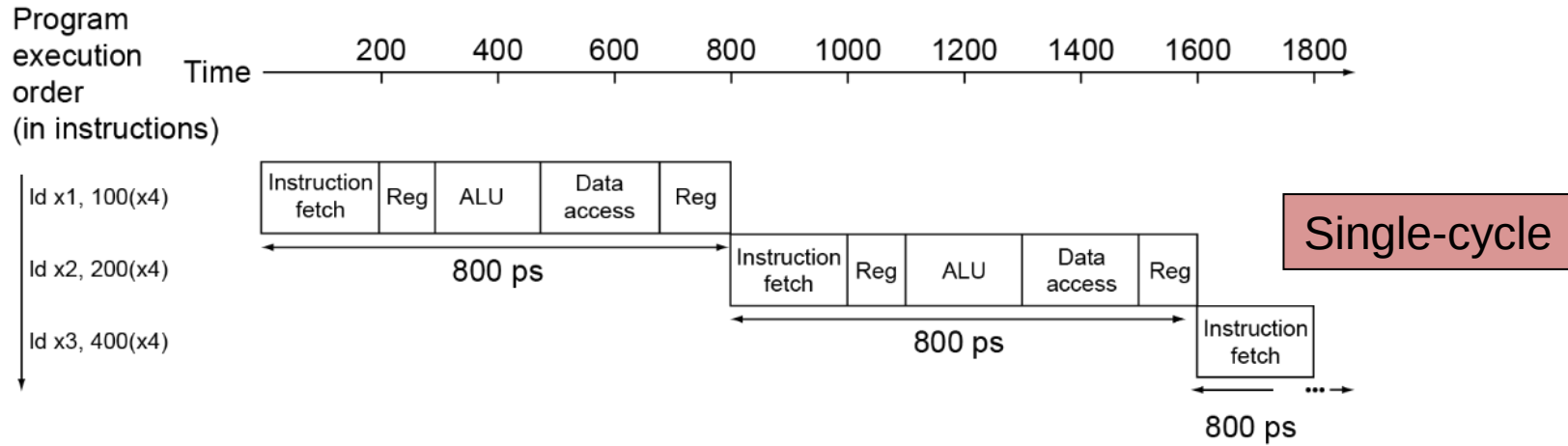
Single-Cycle Implementation

- Clock cycle time must support the longest instruction (i.e. ld)
- Clock Cycle Time : 800 ps

Steps in RISC-V Instruction Execution



RISC-V Performance: Single Cycle



Performance Issues of Single Cycle Processor Design

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

- CPI is 1.
- However, clock cycle time is determined by the longest possible path/delay in the data path.
 - Critical Path: Load Instruction
 - Load instruction uses five functional units in series: Instruction memory → register file → ALU → data memory → register file
- **Although CPI is 1, the overall performance of a single cycle processor implementation is poor because the clock cycle is too long.**

Performance Issues of Single Cycle Processor Design

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

- CPI is 1.
- However, clock cycle time is determined by the longest possible path/delay in the data path.
 - Critical Path: Load Instruction
 - Load instruction uses five functional units in series: Instruction memory → register file → ALU → data memory → register file
- Although CPI is 1, the overall performance of a single cycle processor implementation is poor because the clock cycle is too long.
- We can improve performance by **Pipelining**